

# 第 1 章 绪论

## 1.1 课题背景及研究的目的和意义

蛋白质是生命活动中的核心功能分子，参与催化反应、信号传导、物质运输、免疫识别和细胞结构维持等过程。其功能通常与氨基酸序列、三维结构、局部相互作用模式以及分子结合关系密切相关。传统实验方法可以测定和验证蛋白质结构与功能，但在候选设计、筛选和优化阶段往往周期较长、成本较高。如何借助计算方法生成、筛选和评估候选蛋白质序列，因而成为计算生物学、结构生物学和人工智能交叉研究中的关键问题。

蛋白质计算设计的基本目标，是在给定结构、稳定性、功能或其他属性要求的前提下，生成或优化满足条件的候选序列。早期计算方法主要依赖结构模板、能量函数、构象采样和启发式搜索，Rosetta 等平台体现了此类方法在大分子建模与设计中的代表性工程形态<sup>[1]</sup>。近年来，深度学习方法显著推动了蛋白质结构预测、逆向折叠、序列生成和结构生成的发展。AlphaFold 提高了蛋白质结构预测精度，使结构预测逐渐成为蛋白质设计流程中的重要评估环节<sup>[2]</sup>；AlphaFold 3 将预测范围进一步扩展到蛋白质与核酸、小分子、离子等生物分子相互作用场景<sup>[3]</sup>；ProteinMPNN、ProtGPT2、ESMFold 和 RFdiffusion 等方法分别在逆向折叠、序列生成、快速结构预测和 de novo 结构生成方面形成了可复用能力<sup>[4-7]</sup>。

然而，单个模型能力的提升并不意味着蛋白质设计任务已经实现端到端自动化。一次实际的计算设计任务通常包括目标理解、约束整理、候选序列生成、结构预测、质量检查、目标评分、失败恢复和结果解释等多个环节。不同工具在输入输出格式、运行时间、资源消耗、失败模式和适用边界上存在明显差异。序列生成工具可以扩大候选空间，但不能直接保证目标功能；结构预测工具能够提供关键证据，但通常具有更高计算成本；质量检查和目标评分工具能够过滤低质量候选，却需要根据具体任务选择指标。若系统在任务开始时固定一条工具链并顺序执行，当中间结果质量不足、高代价工具调用失败、预算压力升高或人工确认需求出现时，系统就难以及时依据运行时证据调整后续计划。

与此同时，大语言模型 Agent (Large Language Model Agent, LLM Agent) 为复杂任务自动化提供了任务入口。大语言模型可以理解自然语言目标、生成候选计划、解释中间结果，并通过工具调用扩展自身能力。ReAct 将推理与行动交替组织，使模型在环境反馈中逐步完成任务<sup>[8]</sup>；Toolformer 则表明，语言模型可以学习在适当位置调用外部工具<sup>[9]</sup>。这类工作为自然语言目标解析和工具调用提供了可行实现方式，但蛋白质设计属于高成本、强约束、长链路的科研工作流，仅依靠大语言模型一次性生成计划，仍难以保证计划可执行、过程可恢复、结果可审计。因此，LLM Agent 的任务理解与候选生成能力，还需要同显式工具约束、状态控制、

人工确认和运行时恢复机制结合起来。

本文题为“基于大模型驱动的 agent 协作新一代蛋白质设计系统开发”。本课题的目标，是在已有蛋白质设计工具和大语言模型 Agent 能力的基础上，开发一个面向 de novo 蛋白质设计任务的多 Agent 协作系统，将自然语言设计目标转化为可执行、可追踪、可恢复的工具 workflows。围绕这一目标，本文的价值主要体现在三个方面：在应用层面，系统将分散的蛋白质设计工具组织为统一 workflow，减轻用户在工具选择、参数组织和结果追踪上的负担；在系统工程层面，工具知识图谱、有限状态机、人在环决策和事件日志机制共同提高科研工作流的可控性和可审计性；在算法层面，本文将蛋白质设计流程抽象为受约束、部分可观测、包含高代价步骤的序贯决策问题，并提出适合工程实现的 CEBRA-WP workflow 规划方法，为大模型 Agent 在科学计算任务中的受控协作提供一种可验证路径。

## 1.2 蛋白质设计及其相关技术的发展概述

### 1.2.1 蛋白质计算设计的发展

蛋白质计算设计研究的核心，是根据目标结构或目标功能寻找满足条件的氨基酸序列。由于蛋白质序列空间规模极大，而序列、结构和功能之间的映射关系又具有高度非线性，直接依靠穷举搜索难以完成有效设计。传统方法通常将蛋白质设计问题转化为能量优化、构象采样或结构匹配问题，通过物理化学模型评估候选序列与目标结构之间的相容性。这类方法的优势在于具有明确的结构解释基础，能够与已有生物物理知识结合；不足之处在于搜索空间庞大、计算成本较高，并且在复杂功能、柔性结构和多目标约束场景下需要大量人工经验参与。

随着深度学习模型在蛋白质结构预测和序列建模中的应用，蛋白质计算设计逐渐形成了模型驱动的技术路线。结构预测模型能够根据序列推断三维结构，使设计者可以更快地判断候选序列是否可能折叠为合理构型；逆向折叠模型能够根据给定骨架生成与结构相容的序列；生成式模型能够在更大范围内探索类天然蛋白质序列或结构空间。由此，蛋白质设计不再只依赖单一能量函数和人工迭代，而逐渐表现为多模型、多工具、多指标共同参与的计算流程。

这种发展趋势也带来了新的系统问题。不同模型往往面向不同任务而设计，输入输出形式并不统一，运行所需资源和失败模式也不相同。一个候选序列可能需要经过生成、清洗、结构预测、质量检查和目标评分后才能形成可解释结果；而一旦中间步骤失败，系统需要判断是修补局部参数、替换后续工具，还是终止当前候选并保留已有结果。因此，蛋白质设计技术的发展不仅要求模型本身更准确，也要求系统能够合理组织这些模型，形成可执行和可恢复的工作流。

### 1.2.2 深度学习驱动蛋白质设计工具

从工具类型看，当前蛋白质设计相关模型大致可以分为结构预测、逆向折叠、序列生成和结构生成等方向。结构预测方向以 AlphaFold、AlphaFold 3、ESMFold 和 OpenFold 等工具为代表。AlphaFold 显著提升了结构预测结果的可靠性，使预测结构能够作为候选序列评估的重要证据<sup>[2]</sup>。AlphaFold 3 面向更复杂的生物分子相互作用预测，将应用范围扩展到更复杂的分子系统<sup>[3]</sup>。ESMFold 利用蛋白质语言模型进行结构预测，在速度和部署便利性方面具有优势<sup>[6]</sup>。OpenFold 则提供了 AlphaFold2 的开源实现和工程化再训练基础，为系统集成和本地部署提供了参考<sup>[10]</sup>。

逆向折叠方向主要解决“给定目标骨架，设计相容序列”的问题。ProteinMPNN 使用深度学习从蛋白质骨架结构反推氨基酸序列，在结构条件序列设计中具有代表性<sup>[4]</sup>。序列生成方向则尝试从蛋白质序列分布中学习生成规律。ProtGPT2 将自回归语言模型用于蛋白质序列生成，说明蛋白质序列可以作为符号序列进行无监督建模和候选探索<sup>[5]</sup>。结构生成方向以 RFdiffusion 等方法为代表，扩散模型被用于 de novo 蛋白质结构与功能设计，为从结构空间直接生成候选骨架提供了技术路径<sup>[7]</sup>。

这些工具为本文系统提供了重要基础，但本文的重点并不是重新训练或替代上述模型，而是将这些异构能力纳入统一的工作流系统。对于系统而言，模型能力需要被描述为可调用工具，包括工具名称、输入字段、输出字段、成本等级、风险等级和适用任务。只有在工具能力被结构化表示后，Agent 才能根据用户目标生成候选计划，执行器才能按照统一契约调用工具，安全组件才能在高风险步骤前进行拦截和人工确认。

### 1.2.3 蛋白质设计 workflow 自动化的发展需求

蛋白质设计任务具有明显的工作流特征。一个完整流程往往不是单次模型推理，而是多个工具之间的组合执行：用户的自然语言目标需要转化为结构化约束，候选工具链需要经过输入输出闭包检查，执行过程需要根据中间结果调整后续计划，最终还要汇总序列、结构、指标、风险和日志，形成可复核报告。

传统科学工作流系统强调任务编排、可复现执行和数据追踪，Pegasus、Nextflow 等工作体现了工作流管理在科学计算中的价值<sup>[11,12]</sup>。蛋白质设计任务还带有更强的不确定性和交互性：用户目标常以自然语言表达，初始约束可能不完整；候选工具链并非固定，需要根据预算、质量指标和风险等级动态调整；结构预测、目标评分等步骤成本较高，证据不足时应避免无效调用；高风险或不可逆操作也需要人工确认，不能完全交给自动执行模块决定。

基于这些特点，本文将蛋白质设计 workflow 自动化界定为“受约束的智能工作流规划与执行”问题。系统一方面利用 LLM Agent 的自然语言理解和候选生成能力，

另一方面通过工具知识图谱、有限状态机和人在环机制约束 Agent 行为；自动执行之外，还需要记录事件日志、快照和恢复历史，使执行过程可以被追踪和复核。这一需求构成了本文系统设计与算法设计的直接出发点。

## 1.3 MDP/POMDP 与 Agent 协作相关理论的发展概述

### 1.3.1 马尔可夫决策过程的发展

马尔可夫决策过程 (Markov Decision Process, MDP) 是研究序贯决策问题的重要理论框架。MDP 将决策过程抽象为状态、动作、状态转移和奖励函数，并通过策略描述在不同状态下应选择的动作。Bellman 关于马尔可夫决策过程的工作奠定了动态规划和序贯决策优化的基础<sup>[13]</sup>，Puterman 对离散随机动态规划和 MDP 理论进行了系统整理<sup>[14]</sup>。在完全可观测环境中，MDP 能够描述系统如何根据当前状态选择动作，并通过长期收益优化获得较优策略。

蛋白质设计工作流中也存在类似的序贯决策结构。系统需要在候选生成、结构预测、质量检查、目标评分和恢复处理之间不断选择下一步动作。每一步动作都会改变当前任务状态，并影响后续成本、风险和结果质量。例如，当候选序列质量不足时，系统可以选择局部修补、重新生成候选，或者终止当前路径；当预算消耗较高而证据不足时，系统需要判断是否继续进入结构预测等高代价步骤。因此，MDP 中“状态-动作-转移-收益”的思想为本文理解工作流控制问题提供了基础。

但蛋白质设计工作流难以直接建模为标准 MDP。原因在于，系统并不能完全观察真实状态，也无法准确获得所有状态转移概率和奖励函数。候选序列的真实生物学有效性、后续工具运行是否成功、当前失败原因是否可恢复，往往只能通过中间指标和工具反馈间接判断。这使得本文需要进一步借鉴部分可观测决策理论。

### 1.3.2 部分可观测马尔可夫决策过程的发展

部分可观测马尔可夫决策过程 (Partially Observable Markov Decision Process, POMDP) 将状态不可完全观测的情况纳入决策模型。与 MDP 假设状态可直接获得不同，POMDP 认为系统只能获得与真实状态相关的观测，因此需要维护 belief-state / 信念状态，用以表示系统对隐含状态的估计。Sondik 早期研究了部分可观测马尔可夫过程在无限时域折扣成本下的最优控制问题<sup>[15]</sup>，Monahan 对 POMDP 的理论、模型和算法进行了综述<sup>[16]</sup>，Kaelbling 等进一步系统阐述了部分可观测随机领域中的规划与行动问题<sup>[17]</sup>。

POMDP 对本文的启发在于：在不确定环境中，系统不应只根据单个观测结果作出机械判断，而应综合历史、当前观测和任务目标形成对状态的估计。对应到蛋白质设计工作流中，系统看到的只是工具返回结果、错误信息、质量指标、预算消耗和人工决策，并不能直接知道候选路径是否一定成功。因此，工作流规划器需要

根据这些运行时证据判断继续执行、局部修补、后缀重规划或停止是否更合适。

本文并不求解完整 POMDP。完整 POMDP 需要明确状态空间、动作空间、转移概率、观测概率和奖励函数，而蛋白质设计 workflow 中的工具组合、失败原因、目标偏好和成本约束难以完全形式化。为适应工程实现，本文采用 Lite belief-state / 轻量信念状态近似表示运行时状态，将 evidence sufficiency、risk level、budget pressure 和 recovery need 等特征组织为可记录、可解释的状态摘要。这样既保留了 POMDP 中“在部分观测下依据信念行动”的思想，也避免了完整概率模型难以落地的问题。

### 1.3.3 大语言模型 Agent 与工具协作的发展

大语言模型 Agent 是近年来人工智能应用中的重要方向。与只输出文本的语言模型不同，Agent 更强调在环境中执行任务、调用工具、接收反馈并调整后续行动。ReAct 将推理和行动结合起来，使模型能够在中间观察的反馈下逐步完成复杂任务<sup>[8]</sup>。Toolformer 则从工具调用角度说明，语言模型可以学习在适当位置调用外部工具，以补充自身在计算、检索和外部交互方面的能力<sup>[9]</sup>。这类方法为 LLM Agent 组织蛋白质设计 workflow 提供了直接参照。

但是，科研工作流中的 Agent 协作不能只依赖语言模型自身的生成能力。蛋白质设计任务涉及工具调用、成本控制、安全边界和结果追踪，任何一步错误都可能导致后续结果不可用或不可解释。因此，Agent 系统需要明确组件职责。本文将系统划分为 PlannerAgent、ExecutorAgent、SafetyAgent 和 SummarizerAgent 等角色：PlannerAgent 负责理解目标并生成候选 workflow，ExecutorAgent 负责按照工具契约执行步骤，SafetyAgent 负责识别风险和触发人工确认，SummarizerAgent 负责汇总结果和生成报告。通过角色分离，系统可以减少单一 Agent 同时承担规划、执行、安全审查和总结任务所带来的混乱。

在本文中，MDP/POMDP 理论与 LLM Agent 机制并不是彼此独立的两部分。MDP/POMDP 提供了理解 workflow 动态决策和部分可观测性的理论基础，LLM Agent 则提供自然语言目标解析、候选计划生成和结果解释能力。本文提出的 CEBRA-WP 算法正是在二者之间建立工程连接：它不把大模型输出直接作为最终执行计划，而是通过约束过滤、证据评分、轻量信念状态更新和恢复动作选择，将 Agent 生成的候选 workflow 转化为可执行、可审计、可恢复的系统行为。

## 1.4 本文主要研究内容

围绕“基于大模型驱动的 agent 协作新一代蛋白质设计系统开发”这一题目，本文的研究对象不是某一种新的蛋白质生成模型，而是面向蛋白质设计任务的多 Agent 协作系统、workflow 规划算法和工程实现。本文将已有蛋白质设计工具作为可调用能力，把自然语言目标、结构化约束、工具知识图谱、有限状态机、人在环决策和运行时恢复机制结合起来，构建能够接收用户设计目标、生成候选工具链、执

行蛋白质设计流程、记录运行证据并输出结果报告的系统原型。

本文主要研究内容如下。

(1) 构建面向蛋白质设计工具链的统一任务和工具表示。本文将用户目标、任务约束、工具能力、输入输出字段、成本等级和安全等级纳入结构化契约，并通过面向蛋白质设计的工具知识图谱 (ProteinToolKG) 描述工具之间的可组合关系。该表示方式为候选工作流生成、步骤合法性检查、工具调用和运行时恢复提供基础。

(2) 设计多 Agent 协作的蛋白质设计系统架构。系统采用输入交互、智能规划、工作流执行、安全汇总和资源管理等层次组织功能，并将 PlannerAgent、ExecutorAgent、SafetyAgent 和 SummarizerAgent 的职责分离。系统通过有限状态机 (Finite State Machine, FSM) 约束任务生命周期，通过 PendingAction/Decision 表示人在环决策 (Human-in-the-loop, HITL)，通过事件日志和任务快照记录执行过程。

(3) 提出约束与证据感知、信念引导、恢复自适应工作流规划 (Constraint- and Evidence-aware Belief-guided Recovery-adaptive Workflow Planning, CEBRA-WP) 算法。该算法以目标  $g$ 、约束集合  $C$ 、ProteinToolKG  $K$ 、历史  $h_t$ 、观测  $o_t$  和 Lite belief-state / 轻量信念状态  $x_t$  为输入，经过候选生成、硬可行性过滤、静态评分、后验目标评分、运行时重排序和恢复动作选择，输出候选集合、默认建议和恢复动作。算法支持 continue、patch\_local、suffix\_replan 和 stop 四类动作，并将 stop 映射为受 FSM 和 HITL 约束的终止型重规划候选。

(4) 实现系统原型并开展测试与实验验证。后端采用 Python、FastAPI 和 Pydantic 实现任务接入、计划生成、工作流执行、HITL 接口、事件查询和报告读取；前端采用 React、TypeScript 和 Vite 构建轻量 Web 工作台；工具层通过统一适配器封装 ProtGPT2、ProteinMPNN、ESMFold/OpenFold、Biopython QC、DSSP 和目标评分等能力。本文通过 API 合约、数据契约、FSM 迁移、HITL 决策边界、快照恢复、前端与 CLI 可用性、安全阻断和恢复路径等测试验证系统可用性，并通过 static\_top1、fixed\_threshold\_gate、dynamic\_no\_belief\_state 和 lite\_belief\_state 四组策略对照实验分析 CEBRA-WP 的机制作用。

本文后续章节安排如下：第二章介绍相关技术与理论基础；第三章进行系统需求分析；第四章说明系统设计与 CEBRA-WP 算法定义；第五章介绍系统实现；第六章报告系统测试与验证；第七章分析实验结果；第八章总结全文并讨论后续研究方向。

## 第 2 章 相关技术与理论基础

绪论已经从课题背景、蛋白质设计技术发展、MDP/POMDP 与 Agent 协作理论等角度说明了本文选题的来源。本章转向技术与理论基础的界定，重点解释后续章节反复使用的概念、工具能力和工作流控制依据。由于本文工作位于蛋白质计算设

计、科学工作流、大语言模型 Agent 和运行时恢复控制的交叉处，本章围绕“如何将已有工具组织成可控工作流”展开。

## 2.1 蛋白质计算设计基础

蛋白质由氨基酸序列组成，并在特定条件下折叠为三维结构。通常认为，序列、结构和功能之间存在层级关系：序列影响折叠构象，结构进一步影响结合、催化、稳定性和其他功能属性。蛋白质设计的目标是在给定结构、功能或属性约束下寻找合适序列；de novo 蛋白质设计则进一步强调不直接依赖天然模板，从目标条件出发生成新的候选序列或结构。面向本文系统，蛋白质设计还需要被理解为可拆分、可组合、可恢复的任务过程。

该问题具有高维组合搜索特征。长度为  $n$  的蛋白质序列理论空间为  $20^n$ ，即使实际搜索会受到物理、进化和任务约束限制，候选空间仍远超人工枚举能力。因此，现代蛋白质设计通常将生成、预测、筛选和评分拆分为多个阶段，通过工具链逐步缩小候选空间。

本文采用六阶段能力分层理解 de novo 设计流程：序列探索用于生成候选序列；结构映射用于预测候选序列可能折叠出的三维结构；质量门禁用于剔除长度、残基合法性、低复杂度或结构完整性不满足要求的候选；结构条件精修依据结构反馈重新设计序列；目标评分综合稳定性、结构质量、novelty 和任务目标匹配度对候选排序；结果汇总则生成面向科研人员复核的报告。

这六个阶段不是固定流水线，而是可组合的能力单元。实际任务可能跳过部分阶段，也可能在结构预测与结构条件精修之间迭代。第四章将基于这一能力分层设计系统工作流。

## 2.2 蛋白质设计与分析工具

深度学习和生物信息学工具为本文系统提供底层能力来源。这里按照系统集成视角对工具能力进行归类，说明它们在本文工作流中的角色。

AlphaFold2 显著提升了蛋白质结构预测精度，成为结构预测领域的重要基准<sup>[2]</sup>。AlphaFold3 进一步扩展到更复杂的生物分子相互作用结构预测场景<sup>[3]</sup>。OpenFold 是 AlphaFold2 的开源实现与工程化再训练版本，为结构预测工具接入提供了可部署基础<sup>[10]</sup>。ESMFold 利用蛋白质语言模型表征进行快速结构预测，在速度和部署便利性方面具有优势<sup>[6]</sup>。

在序列设计和生成方面，ProteinMPNN 使用深度学习从蛋白质骨架结构反推氨基酸序列，适合结构条件下的逆向折叠任务<sup>[4]</sup>。ProtGPT2 则以蛋白质语言模型方式生成类天然序列，可用于候选序列探索<sup>[5]</sup>。RFdiffusion 从扩散生成角度推动 de novo 结构与功能设计，为生成式蛋白质设计提供了重要背景<sup>[7]</sup>。

传统平台和基础库同样重要。Rosetta3 长期支持基于物理能量函数的蛋白质建模与设计，是传统蛋白质设计平台的代表<sup>[1]</sup>。Biopython 为序列解析、结构文件处理和基础生物信息学操作提供了稳定接口<sup>[18]</sup>。

这些工具各有优势，但单个工具并不会自动解决跨工具编排问题。结构预测工具关注预测精度，序列设计工具关注候选生成，基础库侧重数据处理。本文系统将这些能力通过 ToolAdapter 和 ProteinToolKG 组织为可组合工具链，并在运行时根据证据、成本和风险选择后续动作。

## 2.3 科学工作流与可复现执行

科学工作流管理系统（Scientific Workflow Management System, SWMS）用于组织复杂计算流程、管理数据依赖、调度任务并记录执行过程。蛋白质设计需要可追踪、可恢复的工作流支撑，因此有必要先明确通用科学工作流系统能够提供的基础能力，以及本文为何没有直接将全局控制流交给通用流程引擎。Pegasus 支持将抽象工作流映射到分布式资源，强调工作流计划、调度和可复现执行<sup>[11]</sup>。Nextflow 以 DSL 描述计算流程，支持容器化、任务缓存和可复现运行，在生物信息学流程中应用广泛<sup>[12]</sup>。

科学工作流系统擅长稳定执行预定义流程并管理计算资源。本文场景中的核心问题不仅是“如何运行一个已定义 DAG”，还包括“何时应改变后续计划”。例如，某个步骤虽然成功执行，但产物质量不足，传统工作流系统通常只能记录步骤完成，难以判断该质量信号对后续蛋白质设计目标的影响。高代价工具失败时，传统恢复也多集中在重试或重新调度，不一定能够判断是否应保留已完成前缀并替换未执行后缀。

因此,本文没有把全局控制流交给外部流程引擎,而是使用自定义 Workflow/FSM 管理任务生命周期。外部工具或流程后端可作为单个 PlanStep 的执行方式接入,但任务创建、候选确认、运行时恢复、HITL（人在环决策）和审计链路由系统自身控制。该选择并不否定科学工作流系统的价值，而是将其定位于执行资源层或单步工具后端，将语义级恢复控制保留在本文系统中。

## 2.4 大语言模型 Agent 与工具调用

大语言模型（Large Language Model, LLM）在自然语言理解、任务分解和工具使用方面表现出较强能力。在本文系统中，LLM Agent 主要承担自然语言入口、候选生成和过程解释等职责。ReAct 将推理与行动结合，使模型能够在外部环境反馈下交替进行思考和工具调用<sup>[8]</sup>。Toolformer 从模型学习角度说明语言模型可以学习在合适时机调用外部工具<sup>[9]</sup>。Tree of Thoughts 强调保留多个候选推理路径并进行搜索，有助于复杂问题求解<sup>[19]</sup>。Reflexion 则利用失败反馈和自我反思改善后续



任务表现<sup>[20]</sup>。

这些研究为本文的候选生成、工具调用和失败反馈机制提供了启发。不过，蛋白质设计工作流不同于一般文本任务：工具调用具有真实计算成本，输出需要满足严格 schema，安全风险和预算约束不能只依赖自然语言判断，任务状态也需要在系统重启和人工等待期间保持一致。OSWorld 等开放环境 Agent 评测工作也提示，Agent 的执行能力、环境稳健性和恢复能力应被显式评估，而不能只评价最终文本输出<sup>[21]</sup>。

本文系统采用多 Agent 职责划分：PlannerAgent 生成候选工具链和解释，ExecutorAgent 调用已确认的工具步骤，SafetyAgent 提供风险信号，SummarizerAgent 汇总最终结果。LLM 可参与候选生成和解释，但候选硬可行性过滤、FSM 状态迁移、HITL 决策应用和运行时恢复动作由结构化算法和系统契约控制。这样可以把 LLM 能力嵌入工作流，而不是让 LLM 独占控制权。

## 2.5 部分可观测规划与预算感知决策

MDP/POMDP 为本文理解运行时恢复提供了理论参照，但本文只借鉴部分可观测决策思想，不求解完整 POMDP。具体到系统执行过程，系统能够观察到步骤成功或失败、错误类型、结构质量指标、安全信号和预算消耗，却无法直接知道当前工具链是否仍然具有全局成功可能，也无法直接区分偶发失败和结构性失败。因此，瞬时观测不足以支持可靠决策。

部分可观测规划研究认为，在环境状态不能被完全观测时，决策者需要维护 belief-state，以历史观测的摘要形式估计当前状态<sup>[17]</sup>。后续部分可观测启发式规划研究进一步强调，启发式函数应考虑随机效应与信息价值<sup>[22]</sup>。本文并不求解完整 POMDP，而是将这一思想工程化为 Lite belief-state / 轻量信念状态：使用 `p_success`、`p_structural_failure`、`recovery_margin`、`expected_remaining_cost` 和 `evidence_sufficiency` 五个变量描述运行时状态。

预算约束同样是高代价科研工作流的重要因素。预算强化学习研究说明，资源消耗和风险约束应被纳入决策目标，而不是在任务结束后再统计<sup>[23]</sup>。在本文场景中，这一思想体现为 CEBRA-WP 对 `expected_remaining_cost`、`budget_pressure` 和高代价调用的显式建模。系统在证据不足时降低高成本候选的排序，在局部失败后优先判断能否通过 `patch_local`（局部修补）保留有效前缀，在恢复余量不足时生成受控的 `stop`（终止型重规划候选）。

## 2.6 相关系统与本文定位

近期蛋白质设计相关预印本中已有工作开始关注多目标搜索、属性引导和系统化设计流程。例如，RosettaSearch 关注蛋白质序列设计中的推理时多目标搜索<sup>[24]</sup>，

AutoBinder Agent 探索面向 binder 设计的端到端 Agent 系统<sup>[25]</sup>，property-driven inverse folding 相关工作则讨论多目标偏好与 developability 约束<sup>[26]</sup>。这些相关预印本提示，蛋白质设计正在从单模型预测走向多目标、多工具和系统化流程。

本文与这些工作的侧重点不同。本文不提出新的蛋白质生成模型，也不声称在湿实验层面验证设计蛋白功能，而是关注 workflow 控制层问题：如何在已有工具之上组织候选 workflow，如何用 ProteinToolKG 约束工具链生成，如何用 Lite belief-state / 轻量信念状态进行运行时重排序，以及如何在失败、风险和预算压力下进行可审计恢复。因此，本文的贡献主要体现为系统架构、workflow 规划算法和工程验证，而不是底层蛋白质模型本身。

图 2-1 给出了本文技术路线涉及的主要技术关系。

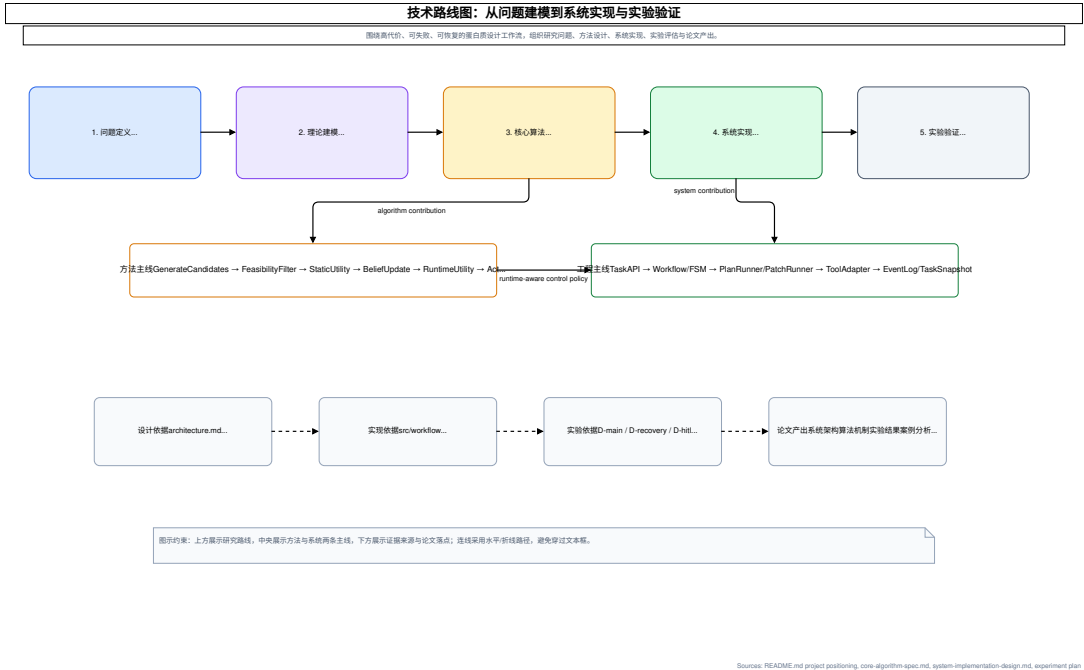


图 2-1 技术路线概览

图 2-1 展示了本文系统与多个技术方向之间的关系：蛋白质设计工具提供计算能力，科学 workflow 提供执行与可复现基础，LLM Agent 提供候选生成和解释能力，部分可观测规划与预算感知决策为 CEBRA-WP 的运行时控制提供理论依据。

## 2.7 本章小结

本章界定了后续系统设计中需要使用的主要技术概念。已有蛋白质工具为序列生成、结构预测和质量评估提供能力来源；科学 workflow 系统为可复现执行提供基础思想；LLM Agent 研究为任务分解、工具调用和失败反馈提供方法参照；部分可观测规划和预算感知决策则支撑运行时恢复控制。这些基础共同指向一个判断：

本文系统需求并非单一工具能力需求，而是跨工具编排、运行时控制和证据追踪的综合需求。下一章将在这些概念界定之上分析系统的具体需求。

## 第3章 需求分析

本文面向的是一个基于大模型驱动的 agent 协作蛋白质设计系统。经过前文对技术背景和理论基础的梳理，本章转入需求分析，从蛋白质设计工作流的实际问题出发，讨论系统需要满足的功能需求、非功能需求和边界约束。与传统单步工具封装不同，本文系统面向高代价、长链路、可失败、可恢复的科研工作流，需求分析除任务管理、接口交互和工具调用外，还需覆盖候选计划生成、HITL（人在环决策）、运行时自适应和审计追踪。

### 3.1 问题界定

本文面对的不是底层蛋白质模型训练问题，而是多工具、多阶段、多状态的工作流控制问题。蛋白质设计任务通常不是一次模型调用即可完成的孤立计算，而是由目标描述、候选序列生成、结构预测、质量筛选、目标评分和结果汇总等环节组成的长链路工作流。不同工具在输入输出格式、运行成本、失败模式和适用边界上存在明显差异：序列生成工具给出候选空间，结构预测工具提供折叠置信度，质量控制与目标评分工具则过滤不可用结果并支持排序。

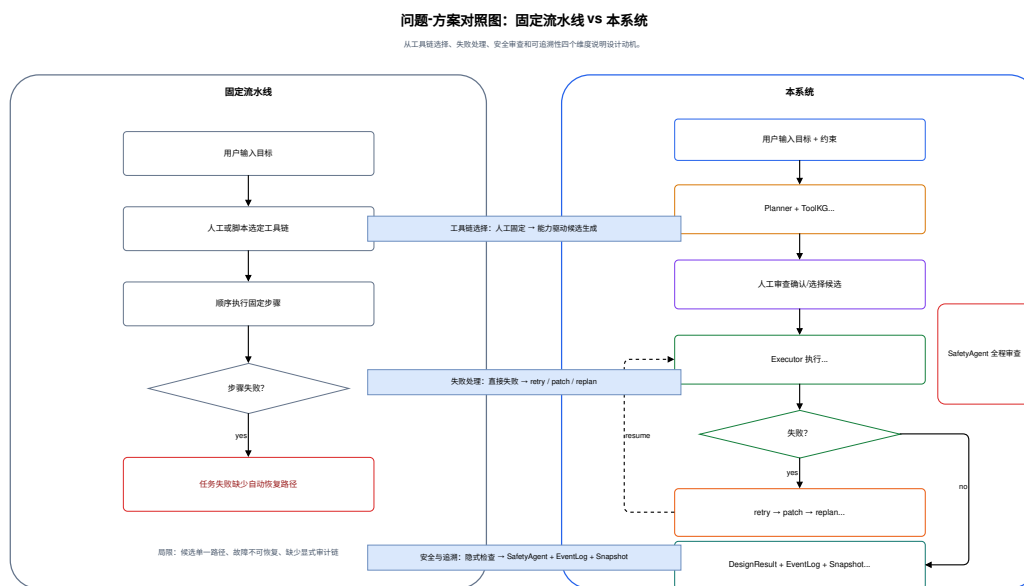
如果仅依赖固定流水线，系统很难在运行中根据中间证据调整后续步骤，也难以在高代价工具失败或结果质量不足时保留已有有效前缀并进行局部修复。科学工作流系统能够支持流程定义、任务调度和可复现执行<sup>[11,12]</sup>，但本文场景还需要对中间结果质量、失败语义、预算压力和人工确认进行语义级处理。开放环境 Agent 评测也说明，Agent 系统的执行、恢复和稳健性需要被明确纳入评估，而不能只关注最终输出<sup>[21]</sup>。

固定流水线与本文系统方案的差异如图 3-1 所示。

图 3-1 左侧表示固定流水线模式：任务开始前由人工或脚本选定工具链，执行过程中一旦步骤失败，通常只能重跑或结束任务。右侧表示本文系统方案：用户目标和约束进入系统后，由 PlannerAgent 与 ProteinToolKG 生成候选工具链，经可行性过滤和人工确认后交由 ExecutorAgent 执行；执行中若出现失败、风险或证据不足，系统按 `retry -> patch_local -> suffix_replan` 和 `terminal_stop` 的层级路径恢复。图中还标出了 SafetyAgent 的风险信号以及 EventLog、TaskSnapshot 形成的审计链，这些机制构成后续系统设计的需求来源。

### 3.2 用户角色与使用场景

系统主要面向两类使用者。



第一类是提交蛋白质设计任务并审查关键决策的科研人员。他们关注任务目标是否被正确解析，候选工具链是否合理，高成本步骤是否值得执行，失败后的修补或重规划是否保留已有有效结果，以及最终报告是否包含必要的序列、结构、指标、风险和审计信息。

第二类是系统开发与实验人员。他们关注工具接入是否规范，工作流失败是否可定位，不同 runtime policy 是否能够复现实验，日志、快照、候选分数、动作效用和实验指标是否完整。

围绕上述用户，系统应支持以下典型场景。

1. 用户提交自然语言设计目标，并补充长度范围、安全等级、结构模板、预算等约束；系统将其转化为结构化任务。
2. 系统根据任务目标和 ProteinToolKG 生成 Top-K 候选计划，并展示默认建议、风险等级、成本估计、解释依据和可能的工具链差异。
3. 当初始计划包含高代价或高风险步骤时，系统进入人工确认状态，等待用户接受、拒绝或选择候选。
4. 执行过程中若出现工具失败、schema 不满足、质量门禁失败、结构预测置信度不足或安全警告，系统根据失败上下文生成局部修补或后缀重规划候选。
5. 任务完成后，系统生成包含序列、结构文件、指标、风险标记和恢复历史的报告，并保留事件日志、快照、决策记录和中间产物路径。

## 3.3 功能需求

### 3.3.1 任务接入与约束确认

系统应提供统一的任务接入能力，将自由文本目标、结构化约束和任务元数据收敛为 `ProteinDesignTask`。蛋白质设计任务通常存在信息不完整的情况，因此任务接入不应只是单次表单提交，而应支持草稿创建、字段补充、场景门控和正式确认。

该模块应满足以下需求。

1. 为每个任务生成唯一 `task_id`，记录目标、约束、创建时间和元数据。
2. 支持自然语言目标、兼容自由文本入口和已确认结构化任务规格，并保证正式任务创建入口语义互斥。
3. 在正式执行前进行场景可行性和工具 `readiness` 预览，避免明显不可执行任务直接进入高代价 workflow。
4. 保留任务草稿、字段补充和确认过程，使后续计划生成能够追溯用户目标与系统解析之间的关系。

### 3.3.2 候选计划生成与工具链选择

系统应根据任务目标、约束集合和 `ProteinToolKG` 中的工具能力信息生成候选计划。候选计划不应只包含工具名称，还应包含步骤依赖、输入引用、输出字段、风险等级、成本估计、评分分解和解释说明。计划的最小执行单元是 `PlanStep`，多个步骤通过 `S{id}.{field}` 形式表达中间结果引用。

候选生成阶段应执行硬可行性过滤。候选中的工具必须存在，输入输出必须闭合，`schema` 必须合法，安全等级与预算硬约束不得被突破，关键工具也应具备可用性信息。对于未触犯硬约束但证据不足或能力降级的候选，系统可以保留为 `degraded feasible`，但必须要求人工确认或明确策略授权。

### 3.3.3 workflow 执行与工具适配

系统应能够按照已确认的 `Plan` 调用具体工具，并将每一步执行结果收敛为统一的 `StepResult`。工具调用通过 `ToolAdapter` 完成，`ExecutorAgent` 只依赖适配器注册表和抽象接口，不直接绑定具体工具内部实现。

执行模块应满足以下需求。

1. 解析步骤输入，包括常量输入和上游步骤输出引用。
2. 统一记录工具身份、适配器身份、执行模式、远程作业 ID、错误类型、指标和产物路径。
3. 对步骤失败进行分类，区分可重试错误、不可重试错误、工具错误、安全阻断和质量门禁失败。

4. 在局部失败时优先遵循 `retry` -> `patch` -> `replan` 的恢复顺序。
5. 允许外部流程后端作为单个 `PlanStep` 的执行方式接入，但多步控制流仍由 `Workflow/FSM/Plan` 负责。

### 3.3.4 HITL 与决策审查

蛋白质设计工作流中存在高风险、高成本和高不确定性的节点，系统不能把所有关键选择都交给自动逻辑。因此，系统需要显式支持 HITL（人在环决策）。在本文系统中，HITL 不是 Agent 临时等待自然语言输入，而是通过 `PendingAction` 和 `Decision` 两个结构化对象完成。

系统至少应支持三类人工确认：初始计划确认、局部修补确认和后缀重规划确认。进入任意 `WAITING_*` 状态前，系统必须持久化候选集合、默认建议、解释信息和任务快照。用户提交 `Decision` 后，系统根据决策类型应用对应的 `Plan`、`PlanPatch` 或 `Replan`，并记录决策事件。对于算法建议的 `stop`，其语义应作为 `terminal_stop` 终止型重规划候选呈现；用户接受后任务进入 `FAILED`，用户主动取消则对应 `CANCELLED`。

### 3.3.5 运行时自适应与恢复决策

固定流程难以充分利用执行过程中的失败码、质量指标、预算消耗和安全信号。因此，系统需要维护 `Lite belief-state` / 轻量信念状态，并在实现中通过 `RuntimeState` 持久化，用于描述当前任务继续执行的成功概率、结构性失败概率、恢复余量、预期剩余成本和证据充分性。

在此基础上，CEBRA-WP 需要支持四类运行时动作：`continue`、`patch_local`、`suffix_replan` 和 `stop`。动作选择不应只依赖某一步是否失败，而应综合阶段、失败类型、重试是否耗尽、安全是否阻断、运行时状态摘要和候选效用。该需求直接服务于高代价步骤治理：在结构预测、结构条件精修或重型目标评分之前，系统应尽量利用低成本证据减少无效调用；在高代价步骤失败后，系统应优先判断是否能够保留前缀并进行局部修补。

### 3.3.6 结果汇总、报告与审计

系统最终输出不仅是单一序列或结构文件，还应包括任务目标、计划版本、步骤结果、结构与质量指标、安全事件、恢复轨迹和报告路径。`SummarizerAgent` 应在任务进入汇总阶段后启动，生成面向用户阅读的 `DesignResult` 和报告文本。

同时，系统需要保留审计链路。事件日志用于记录状态变化、候选生成、人工决策、工具执行和恢复动作；任务快照用于保存执行上下文、已完成步骤、运行时状态和待决策对象。对于长时间运行任务和远程模型调用，该能力是系统可靠性和实验复现的基础。

## 3.4 非功能需求

### 3.4.1 可追溯性

任何计划生成、候选推荐、人工确认、状态迁移和恢复动作都应能从日志或快照中追踪其依据。候选对象中的 `source_refs`、`score_breakdown`、`runtime_adjustment`、`action_utility` 和 `runtime_state_summary` 等字段均应服务于解释与复核。

### 3.4.2 可扩展性

新的蛋白质设计或分析工具应通过 `ToolAdapter` 接入，并在 `ProteinToolKG` 中声明能力、输入输出、成本、风险和可用性信息。`ExecutorAgent` 不应因新增具体工具而改变核心调度逻辑。`PlannerAgent` 应基于工具能力和约束生成候选，而不是把某个工具链写死在代码中。

### 3.4.3 可靠性与可恢复性

任务状态迁移必须受 FSM 约束。进入等待状态前必须完成 `PendingAction` 和快照写入。系统重启后应能够根据最新快照和事件日志恢复执行上下文；恢复到等待态后不应自动推进，以避免绕过人工确认。

### 3.4.4 安全与边界控制

`SafetyAgent` 负责输入、步骤和输出阶段的风险判定，但不直接决定终态或重规划。安全信号应作为 `Planner`、`Workflow` 和 `HITL` 的触发依据。系统不得将计算验证结果扩大为湿实验结论，也不得将未验证指标写成已完成生物学验证。

### 3.4.5 实验可复现性

论文实验需要比较 `static_top1`、`fixed_threshold_gate`、`dynamic_no_belief_state` 和 `lite_belief_state` 等策略组，因此系统应稳定记录 `runtime policy`、候选得分、动作 `utility`、高代价调用次数、恢复成功情况、最终任务状态和运行产物路径。

## 3.5 需求优先级与边界

从论文和系统实现角度看，需求优先级可以分为三层。

第一层是必须满足的控制闭环需求，包括任务接入、候选计划生成、FSM 状态控制、工具执行、`HITL`、事件日志和结果汇总。这些能力构成系统能否端到端运行的基础。

第二层是算法机制需求，包括硬可行性过滤、`RuntimeState` 更新、运行时重排序、`patch_local / suffix_replan` 候选生成和 `stop` 候选语义。这些能力构成 CEBRA-WP 能否落地和验证的基础。

第三层是扩展性需求，包括更多工具接入、更复杂的结构可视化、更大规模任务矩阵和更丰富外部基线。它们有助于系统后续演进，但不应被写成本文已经完成的全部能力。

本文的边界也需要明确：系统不重新实现 AlphaFold、ESMFold、ProteinMPNN 等底层模型，不声称自动验证候选蛋白真实生物功能，也不把机制验证扩大为生产级可靠性证明。本文关注的是工作流层的规划、执行、恢复和审计。

### 3.6 本章小结

本章围绕蛋白质设计工作流在工具组合、运行时调整、人工确认和失败恢复四个方面的功能缺口，归纳出六类功能需求：任务接入与约束确认、候选计划生成与工具链选择、工作流执行与工具适配、HITL 与决策审查、运行时自适应与恢复决策、结果汇总与审计。在非功能层面，系统还需要满足可追溯性、可扩展性、可靠性与可恢复性、安全与边界控制、实验可复现性等要求。

这些需求共同指向一个设计方向：系统应以统一任务契约为入口，以 FSM 为控制骨架，以多 Agent 为职责划分，以 ToolAdapter 和 ProteinToolKG 为工具能力基础，并以 CEBRA-WP 承担运行时规划与恢复控制。下一章将在这一需求约束下展开系统设计。

## 第 4 章 系统设计

本章在需求分析的基础上说明系统总体设计。前一章已经将课题目标细化为任务接入、候选计划生成、工具执行、HITL、运行时恢复和审计追踪等需求。基于这些需求，本文系统设计的重点不是把若干工具串接成固定流水线，而是在可执行约束、安全约束和预算约束下，为蛋白质设计任务提供可规划、可恢复、可审计的工作流运行框架。

本章先介绍系统分层架构、核心组件、有限状态机和六阶段蛋白质设计工作流，再定义约束与证据感知、信念引导、恢复自适应工作流规划（Constraint- and Evidence-aware Belief-guided Recovery-adaptive Workflow Planning, CEBRA-WP）。针对 CEBRA-WP，本章给出其在本文系统中的形式化对象、评分函数、运行时状态、恢复动作和实验可验证性。

### 4.1 设计目标与总体架构

蛋白质设计工作流与一般 Web 后端任务存在明显差异。任务目标具有科学语义，用户可能只给出“设计一个稳定短肽”或“提高候选序列结构质量”等高层目标，系统需要将其转换为可执行步骤。工具调用的成本差异也较大：序列质量检查通常较轻，结构预测、重型打分和重设计则可能带来更高计算成本。与此同时，中



间失败不一定意味着整个任务失败，参数错误、输入引用缺失、质量门禁未通过和外部工具暂时不可用，都可能通过局部修补或后缀重规划恢复。系统输出还需要面向科研场景保留证据链，包括候选生成依据、人工确认记录、执行结果、失败原因和恢复历史。

基于上述特点，系统采用五层分层架构，如图 4-1 所示。

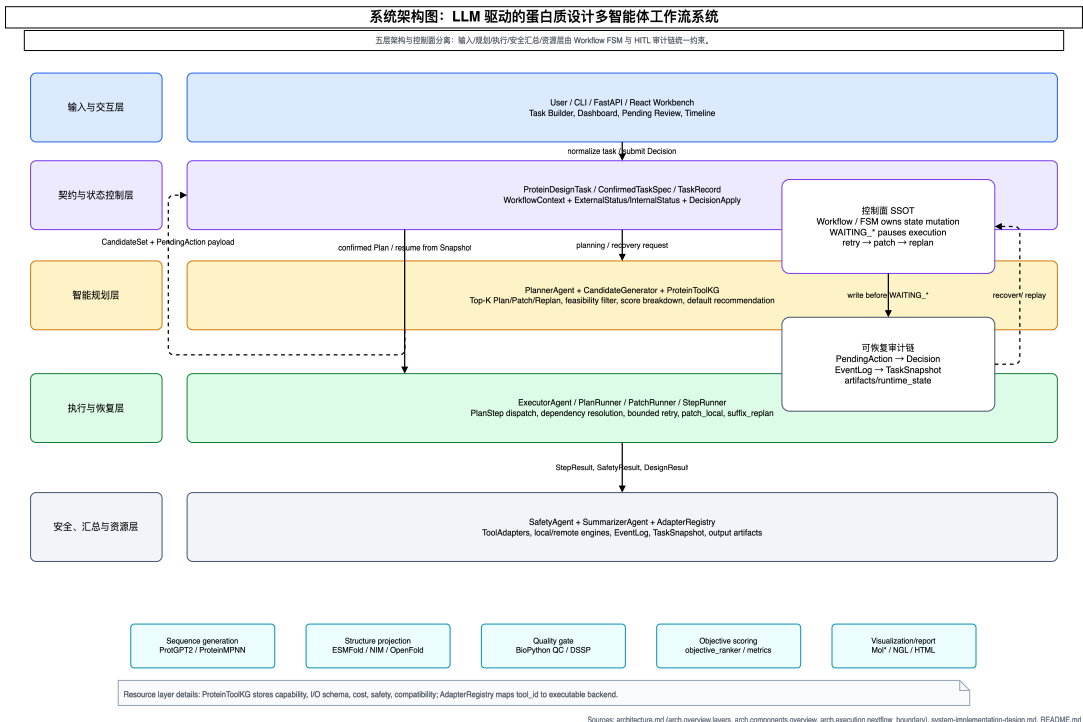


图 4-1 系统五层分层架构

输入层面向 Web 工作台、CLI 和 API，负责接收自然语言目标、结构化约束和人工决策。智能规划层以 PlannerAgent、ProteinToolKG 和 CEBRA-WP 策略为核心，负责生成候选计划、进行可行性过滤、评分和恢复候选生成。执行层由 ExecutorAgent、PlanRunner 和 StepRunner 组成，负责按已确认计划调用工具并记录步骤结果。安全与汇总层包括 SafetyAgent 和 SummarizerAgent，分别负责风险判定与最终报告生成。资源层包括 ToolAdapter 注册表、ProteinToolKG、事件日志、任务快照和文件产物管理。

五层之间通过结构化数据契约交换信息。输入层不直接执行工具，规划层不直接修改任务终态，执行层不越过有限状态机进行状态跳转，安全层不直接编辑计划，汇总层不重新执行计算。通过这种职责拆分，候选生成、工具调用、人工确认和恢复动作都能回到对应的契约对象与事件记录，从而保证复杂任务中的可追溯性。

## 4.2 核心组件与职责边界

系统核心逻辑由四类 Agent 和工具适配层共同完成。

PlannerAgent 负责计划生成和恢复候选生成。它读取任务目标、约束、执行历史和 ProteinToolKG，生成初始 PlanCandidate，或在失败后生成局部修补候选与后续重规划候选。PlannerAgent 的输出必须是结构化候选，而不是无法验证的自然语言建议。每个候选包含候选标识、摘要、结构化载荷、评分分解、风险等级、成本估计、解释文本和来源引用，因此既能被自动策略排序，也能呈现在人工审查界面中。

ExecutorAgent 是工具调用与计划推进组件。它通过 PlanRunner 管理计划级执行流程，通过 StepRunner 处理单个步骤的输入解析、上游引用求解、适配器调用和结果写入。ExecutorAgent 可以识别失败、重试耗尽和安全阻断等信号，但不会自行绕过 Planner 生成修补方案，也不在等待人工确认时继续执行工具。

SafetyAgent 是风险信号源。它在输入、执行过程和输出阶段产生安全判定，输出 ok、warn、block 等等级。warn 可触发人工确认，block 可阻断自动推进并触发重规划候选。SafetyAgent 的定位是风险判定与建议，不负责计划搜索和工具执行。

SummarizerAgent 是结果汇总组件。它读取计划、步骤结果、安全事件和恢复历史，生成面向用户的报告和机器可读的 DesignResult。其输出应反映已完成计算和已有证据，不把未验证推断写成实验结论。

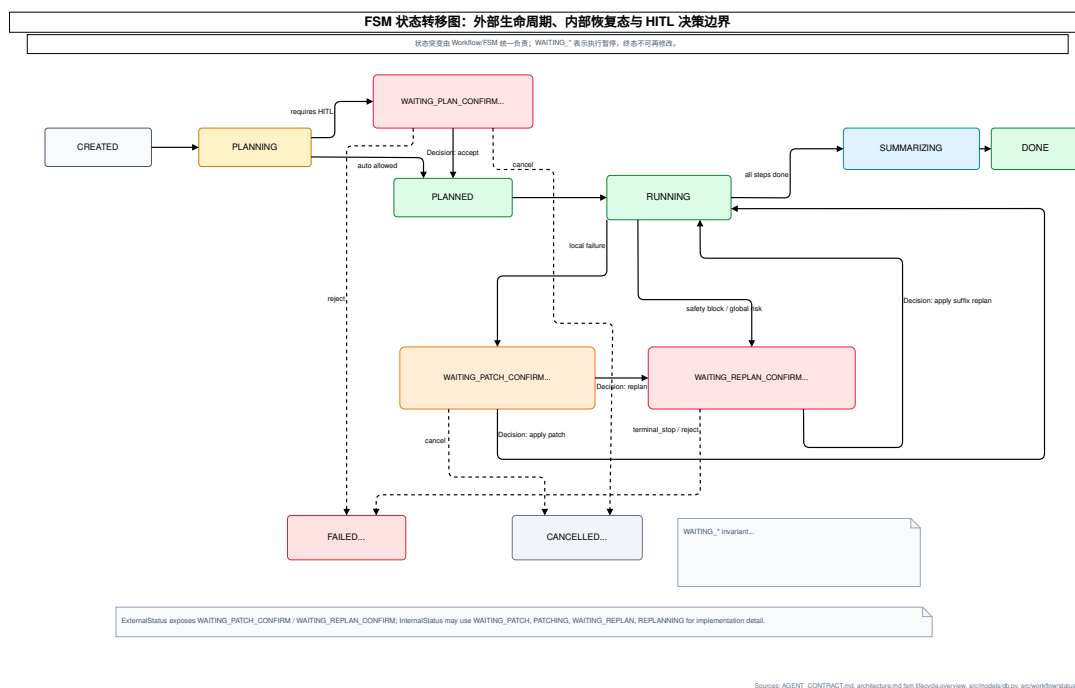
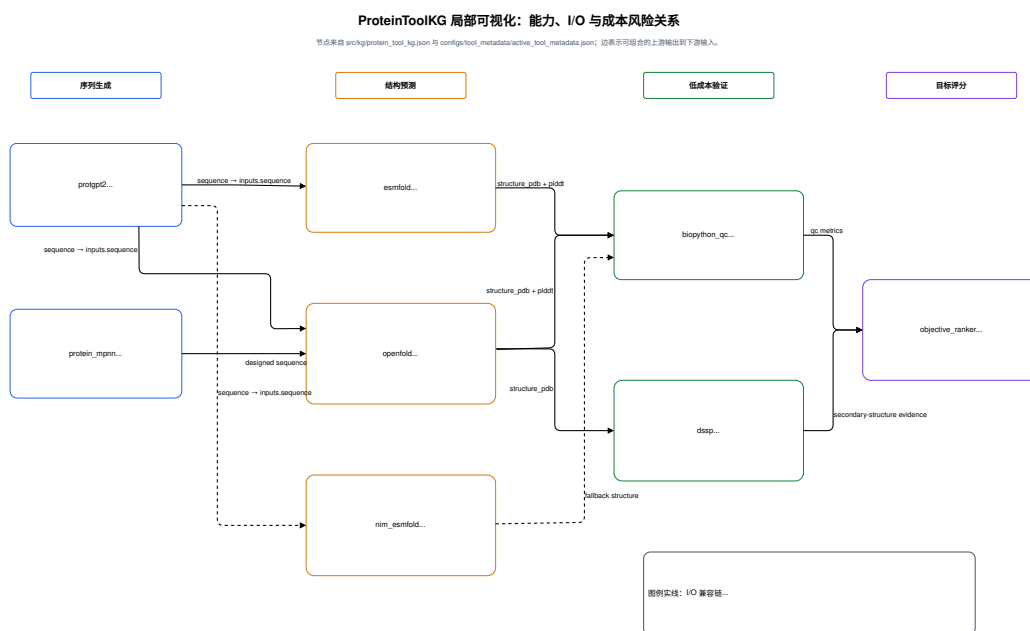
ToolAdapter 层为外部工具提供统一调用接口。工具注册表面向执行层，回答“如何调用工具”；ProteinToolKG 面向规划层，回答“哪些工具可用于何种能力、输入输出如何兼容、成本和风险如何估计”。二者共同支撑候选生成与执行验证。ProteinToolKG 的局部结构如图 4-2 所示。

图 4-2 展示了 ProtGPT2、ProteinMPNN、ESMFold/OpenFold、Biopython QC、DSSP 和 objective\_ranker 等代表性工具之间的能力节点、输入输出字段、成本风险属性和 I/O 兼容边。该图强调本文使用的 ProteinToolKG 是轻量工具能力索引，而非完整生物知识图谱。它的主要作用是为 Planner 提供能力发现、约束校验和候选解释依据。

## 4.3 任务生命周期、FSM 与 HITL 约束

系统使用有限状态机（Finite State Machine, FSM）控制任务生命周期，如图 4-3 所示。

任务对外状态包括 CREATED、PLANNING、WAITING\_PLAN\_CONFIRM、PLANNED、RUNNING、WAITING\_PATCH\_CONFIRM、WAITING\_REPLAN\_CONFIRM、SUMMARIZING、DONE、FAILED 和 CANCELLED。WAITING\_\* 状态具有明确语义：系统已经暂停自动推进，等待人类提交结构化 Decision。DONE、FAILED 和 CAN-



CELLED 为终态，进入后不再被自动修改。

人在环决策（Human-in-the-loop, HITL）不是任意插入的人机交互按钮，而是由成本、风险和不确定性共同触发的受控状态。不同等待状态的进入条件见图 4-4。

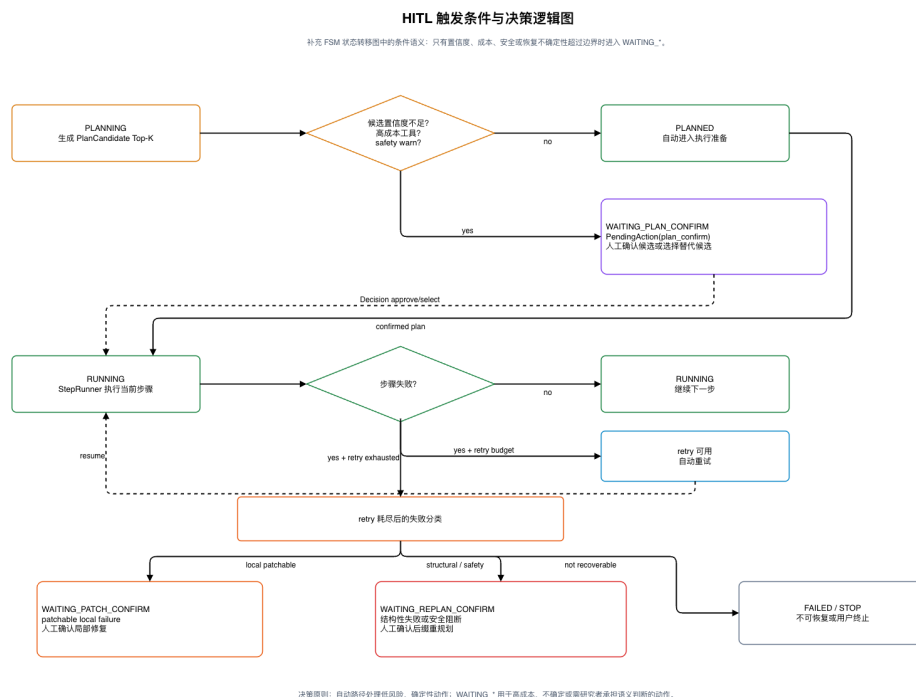


图 4-4 HITL 决策条件

PLANNING 阶段，如果候选置信度不足、即将调用高代价工具或 SafetyAgent 给出 warn，系统进入 WAITING\_PLAN\_CONFIRM。RUNNING 阶段，如果步骤失败但仍有 retry budget，系统先进行有界重试；若重试耗尽且局部可修复，则进入 WAITING\_PATCH\_CONFIRM；若出现结构性失败、安全阻断或恢复余量不足，则进入 WAITING\_REPLAN\_CONFIRM。人工决策通过 Decision 契约提交，Decision Apply 模块验证 pending action 与候选绑定关系，再推动 FSM 合法迁移。

这一设计为后续 CEBRA-WP 算法提供了控制边界：算法可以生成候选、计算分数和提出动作建议，但任务状态仍由 FSM 统一推进；算法建议不能绕过 WAITING\_\* 状态，也不能把 stop 直接变成不可审计的隐式失败。

## 4.4 六阶段蛋白质设计 workflow

蛋白质设计能力被组织为六个阶段，如图 4-5 所示。

六个阶段分别为序列探索、结构映射、质量门禁、结构条件精修、目标评分和结果汇总。序列探索生成候选序列；结构映射将序列转换为结构并产生折叠置信度；质量门禁检查序列合法性、结构完整性和低复杂度等工程质量；结构条件精修根据结构反馈进行重设计；目标评分对候选进行多目标排序；结果汇总生成

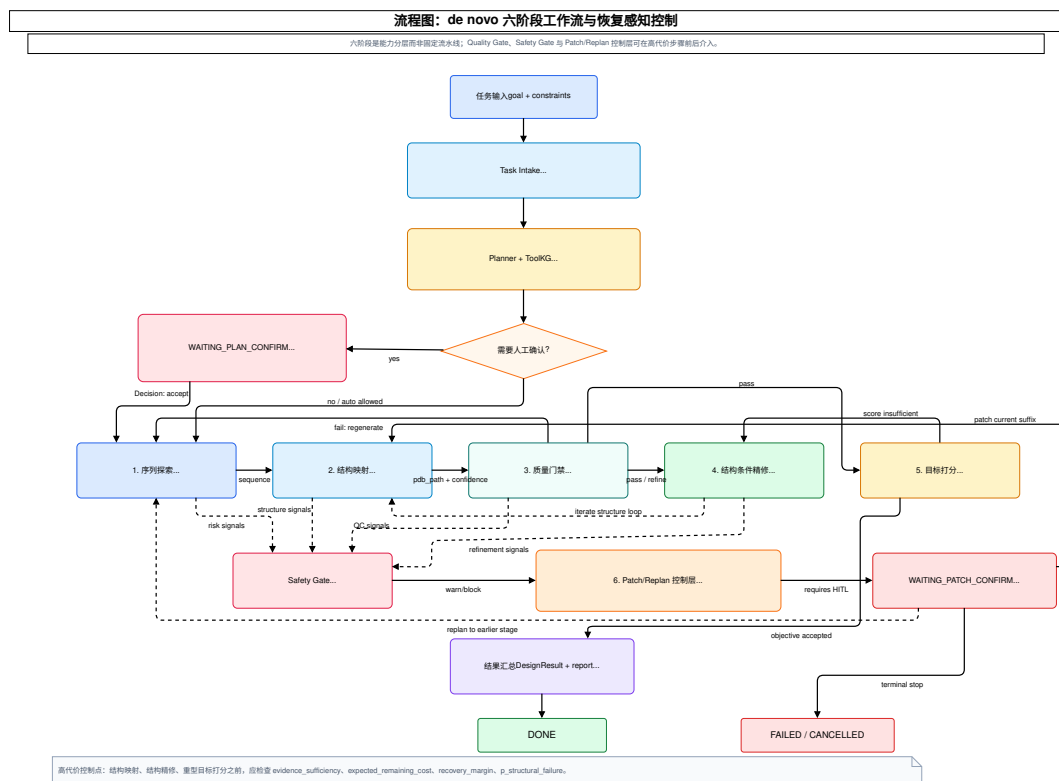


图 4-5 六阶段 de novo 蛋白质设计 workflow

DesignResult 和报告。

六阶段是能力分层，而不是不可改变的线性流水线。质量门禁失败可以回到序列探索，目标评分不足可以回到结构条件精修，结构映射与精修之间可以形成迭代闭环。CEBRA-WP 的作用正是在这些可替代路径之间进行约束化选择：当证据不足时延后高代价步骤，当局部失败可修复时优先 patch\_local（局部修补），当后缀不再可靠时保留有效前缀并进行 suffix\_replan（后缀重规划）。

## 4.5 CEBRA-WP 的提出背景

上一节给出了蛋白质设计 workflow 的能力分层和可替代路径，但阶段划分本身并不能回答何时继续执行、何时局部修补、何时后缀重规划以及何时终止止损。CEBRA-WP 的提出，正是为了处理这些运行时决策。

传统固定流水线适合步骤稳定、成本均匀、失败语义简单的任务，但难以覆盖本文场景。蛋白质设计 workflow 中的关键风险往往具有部分可观测性：一次结构预测失败可能来自工具服务异常，也可能表明上游序列质量不足；一次质量门禁失败可能通过阈值、参数或局部工具替换修复，也可能说明当前候选链路整体失效。系统若只按静态计划执行，会在证据不足时过早调用高代价工具；若只依赖单次 LLM 规划，候选又缺少可复现的约束过滤和恢复依据。

CEBRA-WP 的设计动机主要来自三类研究。部分可观测规划研究指出，决策不应只依赖瞬时观测，而应维护对隐藏状态的 belief-state 表示<sup>[17,22]</sup>。在本文中，隐藏状态不是完整环境模型，而是当前链路是否仍有成功可能、失败是否具有结构性、剩余预算压力和证据是否充分等低维量。预算约束规划与预算强化学习研究强调，成本和风险应作为一等决策变量，而不是执行失败后的附加统计<sup>[23]</sup>；这与蛋白质设计中高代价结构预测和目标评分的治理需求相一致。Tree of Thoughts 强调保留多个候选推理路径并进行搜索<sup>[19]</sup>，Reflexion 则强调利用失败反馈改进后续任务表现<sup>[20]</sup>。本文将这两类思想约束到可执行工具链规划中，使候选、恢复动作和人工确认都以结构化对象表达。

基于这些考虑，CEBRA-WP 被定义为一种面向高代价科研工作流的约束化、证据感知、信念引导、恢复自适应规划算法。它不追求训练端到端最优控制器，而是以可解释、可配置、可审计的形式，把候选生成、硬约束过滤、静态效用、运行时状态、后验证据和恢复动作选择组织为一个闭环。

## 4.6 CEBRA-WP 形式化定义

CEBRA-WP 的完整名称为 Constraint- and Evidence-aware Belief-guided Recovery-adaptive Workflow Planning，本文译为“约束与证据感知、信念引导、恢复自适应的工作流规划”。当前论文版本为 `cebra_wp.v2`，其子公式包括 `static_score.v1`、`posterior_score.v1`、`runtime_adjustment.v1`、`action_utility.v1` 和 `action_bias.v1`。

算法总体闭环如图 4-6 所示。

图 4-6 的上半部分是静态规划层：系统根据目标、约束、ProteinToolKG 和历史记录生成候选工具链，随后执行硬可行性过滤与静态评分。图 4-6 的下半部分是运行时自适应层：系统根据 StepResult、SafetyResult、失败上下文、预算消耗和恢复历史更新 Lite belief-state / 轻量信念状态，再对候选进行运行时重排序，并在 `continue`、`patch_local`、`suffix_replan` 和 `stop` 之间选择动作建议。

算法使用的主要符号如表 4-1 所示。

表 4-1 CEBRA-WP 符号与语义定义

| 符号  | 含义      | 来源或载体                     | 在算法中的作用              |
|-----|---------|---------------------------|----------------------|
| $t$ | 当前决策时间步 | EventLog、RuntimeEvaluator | 索引一次候选生成、状态更新和恢复动作选择 |
| $g$ | 设计目标    | 用户输入、ConfirmedTaskSpec    | 定义任务目标与目标权重          |

表 4-1 (续表)

| 符号                   | 含义                         | 来源或载体                                        | 在算法中的作用                          |
|----------------------|----------------------------|----------------------------------------------|----------------------------------|
| $C$                  | 约束集合                       | 任务约束、策略配置                                    | 限定长度、安全、预算、工具白名单和输出要求            |
| $K$                  | ProteinToolKG              | ProteinToolKG                                | 提供工具能力、I/O schema、兼容关系、成本和风险     |
| $h_t$                | 时间步 $t$ 前的执行历史             | EventLog、TaskSnapshot                        | 提供已完成步骤、失败记录、恢复历史和人工决策           |
| $o_t$                | 当前运行时观测                    | StepResult、SafetyResult、指标、错误细节              | 更新运行时状态并计算后验证据                   |
| $x_t$                | Lite belief-state / 轻量信念状态 | RuntimeState                                 | 表示成功概率、结构性失败概率、恢复余量、剩余成本和证据充分度   |
| $x_{t+1}$            | 观测更新后的轻量信念状态               | RuntimeState 更新结果                            | 为候选重排和恢复动作选择提供更新后状态              |
| $\Pi_{\text{raw},t}$ | 原始候选集合                     | PlannerAgent                                 | 包含初始计划、局部修补或重规划候选                |
| $\Pi_t$              | 过滤后的候选集合                   | FeasibilityFilter                            | 只保留可执行或受保护的 degraded feasible 候选 |
| $\pi$                | 候选 workflow                | PlanCandidate、PatchCandidate、ReplanCandidate | 作为过滤、评分和重排序的基本对象                 |
| $\pi^*$              | 默认候选                       | RuntimeEvaluator、Decision                    | 作为自动执行或 HITL 审查的默认建议             |

表 4-1 (续表)

| 符号                  | 含义      | 来源或载体               | 在算法中的作用                                        |
|---------------------|---------|---------------------|------------------------------------------------|
| $S_{\text{static}}$ | 静态效用    | score_breakdown     | 在运行时观测介入前评价候选先验质量                              |
| $G_{\text{post}}$   | 后验目标匹配  | posterior_objective | 根据证据可靠性修正目标评分                                  |
| $\Delta$            | 运行时修正项  | runtime_adjustment  | 根据状态变量修正候选排序                                   |
| $U_{\pi}$           | 候选运行时效用 | RuntimeEvaluator    | 输出 Top-K 和默认候选                                 |
| $a_t$               | 恢复动作    | action_utility      | 在 continue、patch_local、suffix_replan、stop 之间选择 |

在时间步  $t$ ，算法输入为  $(g, C, K, h_t, o_t, x_t)$ ，输出为结构化 Decision 建议：候选集合  $\Pi_t$ 、默认候选  $\pi^*$ 、候选解释、运行时状态摘要和恢复动作  $a_t$ 。核心计算过程可概括为：

$$\Pi_{\text{raw},t} = \text{GenerateCandidates}(g, C, K, h_t), \quad (4-1)$$

$$\Pi_t = \text{FeasibilityFilter}(\Pi_{\text{raw},t}, C, K, h_t), \quad (4-2)$$

$$S_{\text{static}} = \text{StaticUtility}(\pi, g, C, K), \quad (4-3)$$

$$x_{t+1} = \text{BeliefUpdate}(x_t, o_t, h_t), \quad (4-4)$$

$$G_{\text{post}} = \text{PosteriorObjective}(\pi, g, o_t), \quad (4-5)$$

$$U_{\pi} = \text{RuntimeCandidateUtility}(S_{\text{static}}, G_{\text{post}}, x_{t+1}), \quad (4-6)$$

$$a_t = \text{RecoveryAwareActionSelection}(x_{t+1}, \Pi_t, h_t, C). \quad (4-7)$$

该定义突出两点：一是候选生成与候选选择相互分离，Planner 可以生成多个方案，但只有通过可行性和效用评估的候选才会进入执行或人工确认；二是运行时状态只修正排序和动作建议，不改变 FSM、Agent 职责和硬约束。



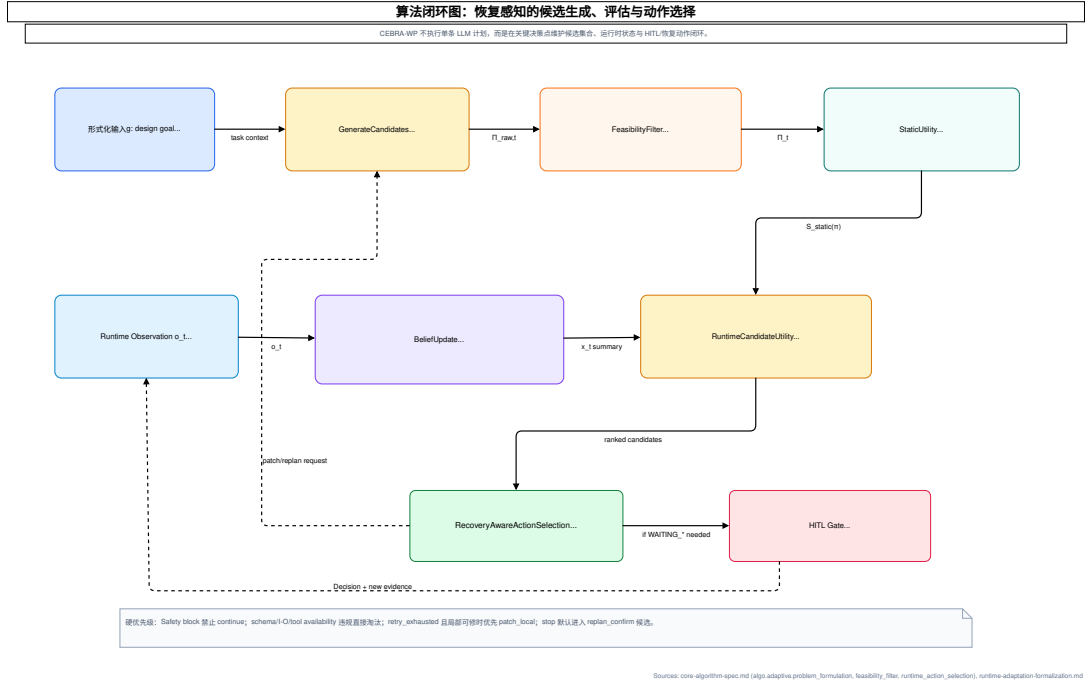


图 4-6 CEBRA-WP 算法闭环

## 4.7 候选生成、硬可行性过滤与静态效用

CEBRA-WP 支持三类候选。PlanCandidate 表示初始完整计划；PatchCandidate 表示对当前计划中局部步骤的参数级、工具级或结构级修补；ReplanCandidate 表示对未执行后缀或整体策略的替换，其中 suffix\_replan 优先保留已验证前缀，terminal\_stop 表示继续投入不划算时的终止型重规划候选。

候选进入评分前必须先通过硬可行性过滤。对候选  $\pi$  定义硬可行性谓词：

$$F_h(\pi, C, K, h_t) = F_{\text{tool}} \wedge F_{\text{schema}} \wedge F_{\text{io}} \wedge F_{\text{safety}} \wedge F_{\text{budget-hard}} \wedge F_{\text{availability}}. \quad (4-8)$$

式中， $F_h$  ——硬可行性谓词；  
 $F_{\text{tool}}$  ——工具存在性约束；  
 $F_{\text{schema}}$  ——输入输出 schema 约束；  
 $F_{\text{io}}$  ——跨步骤 I/O 闭合约束；  
 $F_{\text{safety}}$  ——安全约束；  
 $F_{\text{budget-hard}}$  ——硬预算约束；  
 $F_{\text{availability}}$  ——关键工具可用性或降级路径约束；

其中， $F_{\text{tool}}$  检查候选中的工具是否存在于能力图或适配器注册表； $F_{\text{schema}}$  检查输入输出字段是否满足工具 schema； $F_{\text{io}}$  检查跨步骤引用是否闭合； $F_{\text{safety}}$  检查

是否违反安全等级或触发 safety block;  $F_{\text{budget-hard}}$  检查是否突破不可逾越预算上限;  $F_{\text{availability}}$  检查关键工具是否可用或是否有明确降级路径。过滤后的集合为:

$$\Pi_t = \{\pi \in \Pi_{\text{raw},t} \mid F_h(\pi, C, K, h_t) = 1\}. \quad (4-9)$$

硬可行性过滤承担系统边界约束, 因此静态评分不能把硬不可行候选“打高分后救回”。工程实现可以保留 degraded feasible 候选用于解释或 HITL 审查, 但这类候选必须携带降级原因和人工确认要求。

在候选通过硬过滤后, 算法计算静态效用:

$$\begin{aligned} S_{\text{static}}(\pi) = & w_f F_s(\pi) + w_g G(\pi; g, o_t) - w_c C_{\text{norm}}(\pi) \\ & - w_r R_{\text{norm}}(\pi) - w_{\text{rec}} \text{Rec}(\pi) + w_q Q(\pi). \end{aligned} \quad (4-10)$$

式中,  $F_s(\pi)$  — 候选  $\pi$  的软可行性分数;  
 $G(\pi; g, o_t)$  — 目标匹配度;  
 $C_{\text{norm}}(\pi)$  — 归一化成本;  
 $R_{\text{norm}}(\pi)$  — 归一化风险;  
 $\text{Rec}(\pi)$  — 恢复复杂度;  
 $Q(\pi)$  — 工程可靠性项;  
 $w_f, w_g, w_c$  — 目标、可行性和成本相关权重;  
 $w_r, w_{\text{rec}}, w_q$  — 风险、恢复复杂度和工程可靠性相关权重;

$F_s$  是软可行性分数, 表示候选在 schema 完整、工具 readiness、fallback depth 等方面的先验质量;  $G$  是目标匹配度;  $C_{\text{norm}}$  是归一化成本;  $R_{\text{norm}}$  是归一化风险;  $\text{Rec}$  是恢复复杂度;  $Q$  是工程可靠性项。权重  $w_*$  由策略配置给出, 用于在不同任务设置下调节目标、成本、风险和恢复难度的相对重要性。

静态效用用于形成执行前的候选先验排序。它可以偏好成本更低、风险更小、恢复更容易的工具链, 但无法感知执行中已经发生的失败、证据不足或预算压力变化。因此, 静态效用之后还需要 Lite belief-state / 轻量信念状态和后验证据修正。

## 4.8 Lite belief-state 与观测更新

CEBRA-WP 使用 Lite belief-state / 轻量信念状态近似表示执行过程中无法完全观测的工作流状态。它只保留三类变量: 对恢复动作选择必要的变量、能够从现有日志稳定更新的变量, 以及能够被实验和案例解释的变量。状态向量定义为:

$$x_t = \begin{bmatrix} p_{\text{succ}} \\ p_{\text{sf}} \\ r_{\text{rec}} \\ c_{\text{rem}} \\ e_{\text{suf}} \end{bmatrix}. \quad (4-11)$$

式中,  $p_{\text{succ}}$  ——任务继续成功概率;  
 $p_{\text{sf}}$  ——结构性失败概率;  
 $r_{\text{rec}}$  ——恢复余量;  
 $c_{\text{rem}}$  ——预期剩余成本;  
 $e_{\text{suf}}$  ——证据充分度;

各状态量的语义如表 4-2 所示。

表 4-2 Lite belief-state / 轻量信念状态变量

| 状态量                       | 取值范围     | 含义                       | 主要更新来源                       | 决策作用                                 |
|---------------------------|----------|--------------------------|------------------------------|--------------------------------------|
| $p_{\text{succ}}$<br>成功概率 | $[0, 1]$ | 当前链路继续执行后完成任务的估计概率       | 步骤成功、质量指标、候选静态分、失败记录         | 支持 continue 与 stop 判断                |
| $p_{\text{sf}}$<br>结构失败   | $[0, 1]$ | 当前链路遭遇结构性失败或后续必须重规划的估计概率 | 结构预测失败、质量门禁失败、安全阻断、重复失败      | 提高 patch_local、suffix_replan、stop 权重 |
| $r_{\text{rec}}$<br>恢复余量  | $[0, 1]$ | 在保留有效前缀前提下继续恢复的余量        | 已完成步骤比例、失败类型、patch/replan 次数 | 区分局部修补与后缀重规划                         |
| $c_{\text{rem}}$<br>剩余成本  | 非负实数     | 从当前状态到任务结束的剩余成本暴露        | 剩余步骤、工具成本先验、预算配置、重试记录        | 派生预算压力并约束高代价动作                       |
| $e_{\text{suf}}$<br>证据充分  | $[0, 1]$ | 当前证据是否足以支持进入更高代价步骤       | 质量门禁、结构指标、目标评分、证据可靠性         | 控制高代价步骤推进与人工确认                       |

运行时观测  $o_t$  来自 StepResult、SafetyResult、失败上下文、局部修补/重规划历史、已完成步骤、剩余后缀和 HITL 决策记录。派生量如  $b_{\text{press}}$ 、 $v_{\text{hitl}}$ 、 $\ell_{\text{patch}}$  和  $p_{\text{pres}}$

不作为持久化主状态，而是根据当前状态和候选上下文按需计算。这样可以避免把与具体候选强相关的临时判断写入长期状态，从而降低状态漂移风险。

以预算压力为例， $c_{\text{rem}}$  保留剩余成本原始估计，不直接等同于  $[0, 1]$  区间内的预算压力。若任务给出预算上限  $\text{budget\_cap}$ ，则：

$$b_{\text{press}} = \text{clip} \left( \frac{c_{\text{rem}}}{\max(c_{\text{cap}}, 0.1)}, 0, 1.5 \right). \quad (4-12)$$

式中， $b_{\text{press}}$  ——预算压力；  
 $c_{\text{cap}}$  ——任务预算上限；  
 $\text{clip}(\cdot)$  ——裁剪函数；  
 若任务未给出预算上限，则使用：

$$b_{\text{press}} = \text{clip}(c_{\text{rem}}, 0, 1.5). \quad (4-13)$$

因此，预算压力是由任务上下文派生的动作决策量，而不是 `RuntimeState` 的持久化字段。该区分使系统既能记录可解释的剩余成本，也能在不同预算设置下统一比较恢复动作。

状态初始化可由首选候选的静态分数、风险分数、恢复复杂度、剩余成本先验和廉价证据覆盖率给出。例如， $p_{\text{succ}}$  可由静态候选分数裁剪得到， $p_{\text{sf}}$  可由风险项裁剪得到， $r_{\text{rec}}$  可由恢复复杂度的反向量估计， $c_{\text{rem}}$  来自剩余工具成本先验， $e_{\text{suf}}$  来自低成本质量证据覆盖率。随着执行推进，成功步骤会提升  $p_{\text{succ}}$  和证据充分度，结构性失败会提高  $p_{\text{sf}}$  并降低恢复余量，重复失败和预算消耗则会提高剩余成本压力。

该状态设计吸收了部分可观测规划中 `belief-state` 的思想<sup>[17]</sup>，但在工程上保持轻量：系统不求解完整 POMDP，也不在线训练策略网络，而是使用可解释变量为候选重排和恢复动作提供一致依据。

## 4.9 后验目标评分与运行时重排序

蛋白质设计目标通常是多目标的，包括结构质量、稳定性、功能、`novelty` 和安全性等。不同目标的证据来源不同，证据可靠性也不同。为避免把弱证据与直接证据等价处理，CEBRA-WP 使用证据加权后验目标匹配：

$$G_{\text{post}}(\pi; g, o_t) = \sum_m \lambda_m(g) \rho_m(o_t) q_m(\pi, o_t). \quad (4-14)$$

式中,  $m$  —目标维度;  
 $\lambda_m(g)$  —由任务目标  $g$  决定的维度权重;  
 $\rho_m(o_t)$  —观测  $o_t$  对目标维度  $m$  的证据可靠性权重;  
 $q_m(\pi, o_t)$  —候选  $\pi$  在目标维度  $m$  上的归一化分数;

其中,  $m$  表示目标维度,  $\lambda_m(g)$  是由任务目标决定的目标权重,  $q_m(\pi, o_t)$  是候选在该维度上的归一化分数,  $\rho_m(o_t)$  是证据可靠性权重。证据状态分为 direct、proxy、degraded 和 missing: direct evidence 指已产生的结构质量指标, proxy evidence 指由轻量质量门禁推断出的间接信号, degraded evidence 表示工具降级或证据覆盖不完整, missing 表示当前没有可用证据。整体证据充分度可写为:

$$e_t = \text{clip} \left( \sum_m \lambda_m(g) \rho_m(o_t), 0, 1 \right). \quad (4-15)$$

式中,  $e_t$  —时间步  $t$  的整体证据充分度;

该值进入  $e_{\text{suf}}$ , 影响是否继续进入高代价步骤。若证据不足而候选下一步成本较高, 运行时重排序会降低该候选; 若已有充分低成本证据支持, 则候选可以更合理地进入结构预测或目标评分。

将  $G_{\text{post}}$  替换静态效用中的目标匹配项后, 得到包含后验证据的基础分  $S_{\text{post}}$ 。运行时重排序定义为:

$$U_\pi(\pi, x_t) = \text{clip} (S_{\text{post}}(\pi) + \Delta(\pi, x_t), 0, 1). \quad (4-16)$$

式中,  $S_{\text{post}}(\pi)$  —替换后验目标项后的基础分;

其中  $\Delta(\pi, x_t)$  是有界运行时修正项, 取值范围控制在  $[-0.35, 0.35]$ 。其一般形式为:

$$\begin{aligned} \Delta(\pi, x_t) = & k_s (p_{\text{succ}} - 0.5) \text{Conf}(\pi) \\ & + k_e (2e_{\text{suf}} - 1) \max(\text{Conf}(\pi), F_s(\pi)) \\ & - k_f p_{\text{sf}} (1 - \text{RiskScore}(\pi)) \\ & + k_r r_{\text{rec}} \text{RecoveryScore}(\pi) \\ & - k_c b_{\text{press}} (1 - \text{CostScore}(\pi)) \\ & + k_a \text{ActionBias}(\pi, x_t). \end{aligned} \quad (4-17)$$

|                               |                     |
|-------------------------------|---------------------|
| 式中, $k_s, k_e, k_f$           | ——成功、证据和失败相关修正权重;   |
| $k_r, k_c, k_a$               | ——恢复、成本和动作偏好相关修正权重; |
| $\text{Conf}(\pi)$            | ——候选置信度;            |
| $\text{RiskScore}(\pi)$       | ——候选风险质量分数;         |
| $\text{RecoveryScore}(\pi)$   | ——候选恢复友好度;          |
| $\text{CostScore}(\pi)$       | ——候选成本质量分数;         |
| $\text{ActionBias}(\pi, x_t)$ | ——候选与当前恢复动作偏好的匹配程度; |

其中,  $\text{Conf}(\pi)$  表示候选置信度,  $\text{RiskScore}(\pi)$  和  $\text{CostScore}(\pi)$  为候选的风险与成本质量分数,  $\text{RecoveryScore}(\pi)$  表示候选恢复友好度,  $\text{ActionBias}(\pi, x_t)$  表示候选与当前恢复动作偏好的匹配程度。其直观含义是: 当  $p_{sf}$  和预算压力升高时, 算法降低高成本、低可恢复候选的排序; 当  $p_{succ}$ 、 $r_{rec}$  和  $e_{suf}$  较高时, 算法提高继续执行或低风险候选的排序。

运行时重排序有两个边界: 一是只作用于已经通过可行性校验的候选, 不能覆盖工具不存在、schema 错误、I/O 不闭合和安全阻断; 二是修正幅度存在上界, 避免一次异常观测完全覆盖静态目标匹配和工程可靠性判断。借助这两个边界, 算法在适应运行时变化的同时仍能保持可复现和可审计。

## 4.10 恢复动作选择与 HITL 映射

CEBRA-WP 将恢复动作限定为四类: `continue`、`patch_local`、`suffix_replan` 和 `stop`。四类动作与 FSM/HITL 的映射如表 4-3 所示。

表 4-3 恢复动作与 FSM/HITL 映射

| 动作                         | 触发背景                     | 系统效果                      | FSM/HITL 映射                               | 审计结果                    |
|----------------------------|--------------------------|---------------------------|-------------------------------------------|-------------------------|
| <code>continue</code>      | 成功概率较高、证据充分、风险和预算压力可接受   | 继续执行当前计划后续步骤              | RUNNING 内自动推进<br>或维持 PLANNED / RUNNING    | 记录动作效用与继续原因             |
| <code>patch_local</code>   | 局部失败、重试耗尽、前缀仍有效、局部可修复性较高 | 生成局部 PlanPatch, 替换参数或局部工具 | WAITING_PATCH_CONFIRM<br>或策略允许的受控自动 patch | 记录 patch 候选、失败上下文和应用结果  |
| <code>suffix_replan</code> | 结构性失败概率升高、后缀可靠性不足、前缀可保留  | 保留已验证前缀, 替换未执行后缀          | WAITING_REPLAN_CONFIRM                    | 记录 replan 候选、前缀保留位置和新后缀 |

表 4-3 (续表)

| 动作   | 触发背景                                   | 系统效果                         | FSM/HITL 映射                                   | 审计结果                                 |
|------|----------------------------------------|------------------------------|-----------------------------------------------|--------------------------------------|
| stop | 成功概率低、预算<br>压力高、恢复余量<br>低、人工介入价值<br>有限 | 生成终止型<br>Replan<br>Candidate | WAITING_REPLAN<br>_CONFIRM<br>接受后进入<br>FAILED | 记录<br>terminal<br>_reason 与已<br>保留证据 |

动作效用由状态量和派生量共同计算。为便于表示，记  $s = p_{\text{succ}}$ ,  $f = p_{\text{sf}}$ ,  $r = r_{\text{rec}}$ ,  $e = e_{\text{suf}}$ ,  $b = b_{\text{press}}$ 。局部可修复性、证据可复用性、前缀可保留性、预算缓解度、目标重对齐收益和人工介入价值，分别记为  $\ell_{\text{patch}}$ 、 $e_{\text{reuse}}$ 、 $p_{\text{pres}}$ 、 $b_{\text{relief}}$ 、 $g_{\text{align}}$  和  $v_{\text{hitl}}$ 。动作效用可写为：

$$U_{\text{continue}} = 0.38s + 0.14e + 0.12r - 0.22f - 0.14b, \quad (4-18)$$

$$U_{\text{patch\_local}} = 0.20s + 0.24r + 0.18\ell_{\text{patch}} + 0.12e_{\text{reuse}} - 0.14f - 0.12b, \quad (4-19)$$

$$U_{\text{suffix\_replan}} = 0.18(1 - s) + 0.20f + 0.16(1 - r) + 0.18p_{\text{pres}} + 0.14b_{\text{relief}} + 0.14g_{\text{align}}, \quad (4-20)$$

$$U_{\text{stop}} = 0.32(1 - s) + 0.24b + 0.18(1 - r) + 0.16s_{\text{safe}} + 0.10(1 - v_{\text{hitl}}). \quad (4-21)$$

|                             |                         |
|-----------------------------|-------------------------|
| 式中, $U_{\text{continue}}$   | ——继续执行动作效用;             |
| $U_{\text{patch\_local}}$   | ——局部修补动作效用;             |
| $U_{\text{suffix\_replan}}$ | ——后缀重规划动作效用;            |
| $U_{\text{stop}}$           | ——终止型重规划动作效用;           |
| $s, f, r$                   | ——成功概率、结构性失败概率和恢复余量简写量; |
| $e, b$                      | ——证据充分度和预算压力简写量;        |
| $\ell_{\text{patch}}$       | ——局部可修复性;               |
| $e_{\text{reuse}}$          | ——证据可复用性;               |
| $p_{\text{pres}}$           | ——前缀可保留性;               |
| $b_{\text{relief}}$         | ——预算缓解度;                |
| $g_{\text{align}}$          | ——目标重对齐收益;              |
| $s_{\text{safe}}$           | ——安全终止性;                |
| $v_{\text{hitl}}$           | ——人工介入价值;               |

这组效用函数体现了四类动作的不同偏好: `continue` 对较高成功概率、证据充分度和恢复余量更敏感; `patch_local` 更依赖局部可修复性、证据可复用性和较低结构性失败概率; `suffix_replan` 偏向于结构性失败概率较高、当前成功概率较低但前缀仍可保留的场景; `stop` 只有在继续价值低、预算压力高、恢复余量低且人工介入价值不足时才会成为强候选。

`stop` 不是用户主动取消, 也不是执行异常崩溃, 而是算法提出的终止型重规划候选。其载体为 `ReplanCandidate(replan_mode="terminal_stop")`, 通过 `replan_confirm` 通道进入 `WAITING_REPLAN_CONFIRM`。只有在人工接受或策略显式允许时, 系统才将任务推进到 `FAILED`; 已完成前缀、产物和解释字段仍被保留为审计资产。

综合上述设计, CEBRA-WP 主流程可写为算法 4-1。

#### 算法 4-1 CEBRA-WP 主流程

**Input:**  $g$ : 设计目标;  $C$ : 约束集合;  $K$ : ProteinToolKG;  $h_t$ : 执行历史;  $o_t$ : 当前观测;  $x_t$ : Lite belief-state。

**Output:**  $\text{Decision}_t$ : 候选集合、默认候选、恢复动作、解释和证据引用。

1.  $\Pi_{\text{raw},t} \leftarrow \text{GenerateCandidates}(g, C, K, h_t)$
2.  $\Pi_t \leftarrow \text{FeasibilityFilter}(\Pi_{\text{raw},t}, C, K, h_t)$
3. if  $\Pi_t$  is empty and `degraded_feasible` candidates exist:
4.    $\Pi_t \leftarrow \text{degraded\_feasible}$  candidates
5.   mark candidates as requiring HITL confirmation
6. for each  $\pi \in \Pi_t$ :



```

7.  $S_{\text{static}}(\pi) \leftarrow \text{StaticUtility}(\pi, g, C, K)$ 
8.  $S_{\text{post}}(\pi) \leftarrow \text{ReplaceObjectiveTerm}(S_{\text{static}}(\pi),$ 
    $G_{\text{post}}(\pi; g, o_t))$ 
9.  $x_{t+1} \leftarrow \text{BeliefUpdate}(x_t, o_t, h_t)$ 
10. for each  $\pi \in \Pi_t$ :
11.  $\Delta(\pi, x_{t+1}) \leftarrow \text{RuntimeAdjustment}(\pi, x_{t+1})$ 
12.  $U_\pi(\pi, x_{t+1}) \leftarrow \text{clip}(S_{\text{post}}(\pi)$ 
    $+ \Delta(\pi, x_{t+1}), 0, 1)$ 
13.  $\text{TopK}_t \leftarrow \text{SelectDiverseTopK}(\Pi_t, U_\pi, \text{capability\_coverage})$ 
14.  $U_a \leftarrow \text{ComputeActionUtility}(\text{continue}, \text{patch\_local}, \text{suffix\_replan}, \text{stop})$ 
15.  $a_t \leftarrow \text{ApplyHardPrioritiesAndSelectAction}(U_a, x_{t+1}, h_t, C)$ 
16. return  $\text{Decision}_t(\text{TopK}_t, a_t, \text{explanations}, \text{evidence\_refs})$ 

```

算法第 1 至 8 行生成并评价候选，第 9 至 12 行根据运行时状态修正候选效用，第 13 行保留 Top-K 多样性，第 14 至 15 行选择恢复动作。第 15 行的硬优先级包括安全阻断优先、schema/I/O/tool availability 违规淘汰、重试耗尽后优先局部修补、前缀可保留时优先后缀重规划等约束。借助这些优先级，动作选择遵循系统控制边界，而不是只按连续效用值排序。

## 4.11 策略组与实验可验证性

为验证 CEBRA-WP 各组成机制的作用，系统将算法能力拆分为四组策略开关。四组策略并非四套不同系统，而是在同一工程实现上逐步打开静态评分、固定阈值门控、动态恢复和 Lite belief-state 的内部消融配置，如表 4-4 所示。

表 4-4 四组消融策略与算法机制开关

| 策略组                               | 静态<br>评分 | 固定<br>阈值<br>门控 | 动态<br>观测 | Lite<br>belief-<br>state | Runtime<br>rerank | 恢复<br>动作<br>效用 | 主要实验问题           |
|-----------------------------------|----------|----------------|----------|--------------------------|-------------------|----------------|------------------|
| <code>static_top1</code>          | 启用       | 未启用            | 未启用      | 未启用                      | 未启用               | 未启用            | 单一静态最优候选是否足以完成任务 |
| <code>fixed_threshold_gate</code> | 启用       | 启用             | 有限使用     | 未启用                      | 未启用               | 未启用            | 固定阈值是否能控制风险与成本   |

表 4-4 (续表)

| 策略组             | 静态<br>评分 | 固定<br>阈值<br>门控 | 动态<br>观测 | Lite<br>belief-<br>state | Runtime<br>rerank | 恢复<br>动作<br>效用 | 主要实验问题                          |
|-----------------|----------|----------------|----------|--------------------------|-------------------|----------------|---------------------------------|
| dynamic         |          |                |          |                          |                   |                |                                 |
| _no_belief      | 启用       | 启用             | 启用       | 未启用                      | 部分<br>启用          | 部分<br>启用       | 直接运行时观测是<br>否已能支撑恢复             |
| _state<br>_lite |          |                |          |                          |                   |                |                                 |
| _belief         | 启用       | 启用             | 启用       | 启用                       | 启用                | 启用             | 显式状态是否带来<br>更好的解释、预算<br>感知和恢复决策 |
| _state          |          |                |          |                          |                   |                |                                 |

该策略设计使第 7 章实验能够分别回答三个问题：静态计划是否足够；固定阈值门控是否会带来额外成本；在动态恢复已经存在时，Lite belief-state 的增量价值体现在哪里。根据已有实验设置，本文不把 CEBRA-WP 论证为所有指标上的性能最优算法，而是重点验证其机制链路是否可执行、状态是否可观测、成本控制是否具有方向性证据，以及恢复决策是否具有可解释依据。

## 4.12 数据契约与模块协作

系统以统一数据契约支撑算法与工程实现之间的衔接。核心契约关系如图 4-7 所示。

ProteinDesignTask 是任务入口契约，包含 task\_id、goal 和约束字段。Plan 由多个 PlanStep 组成，步骤之间通过 S{id}. {field} 引用语法建立数据依赖。StepResult 记录每个步骤的执行状态、输出、指标、产物路径和失败信息。PendingAction 与 Decision 构成 HITL 的双向契约：前者承载候选集合、默认建议和解释，后者记录用户选择、决策人和时间戳。TaskSnapshot 保存计划版本、已完成步骤、pending\_action\_id 和 runtime\_state 摘要。RuntimeState 持久化 CEBRA-WP 的五个核心状态量。DesignResult 是最终输出契约，包含序列、结构路径、评分、风险标记、报告路径和元数据。

图 4-8 给出了典型任务 t1 的贯穿示例。

该示例从任务创建开始，经 Planner 生成候选计划、用户确认、Executor 调用工具、StepResult 写入、RuntimeState 更新和必要的恢复候选触发，最后由 Summarizer 汇总为 DesignResult。CEBRA-WP 在这一过程中并非孤立算法模块，而是嵌入任务生命周期的规划与恢复层：算法输出通过 PendingAction、PlanPatch、ReplanCandidate 和 RuntimeState 等契约进入系统，系统状态则仍由 FSM 和 Workflow 控制。

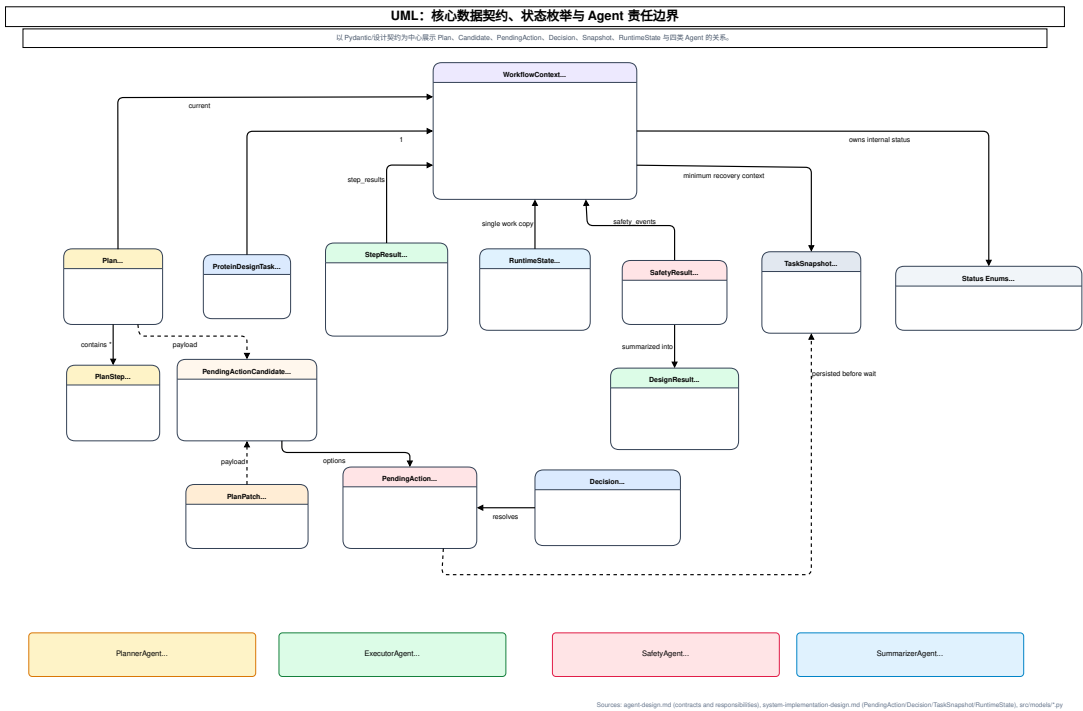


图 4-7 核心数据契约 UML

#### 具体实例走查：t1\_trpcage\_denovo\_short\_peptide 数据流

基于 thesle-final-v1-001 的 Trp-cage-like 短肽任务配置与运行结果，展示任务对象、计划执行和结果汇总之间的字段传递。

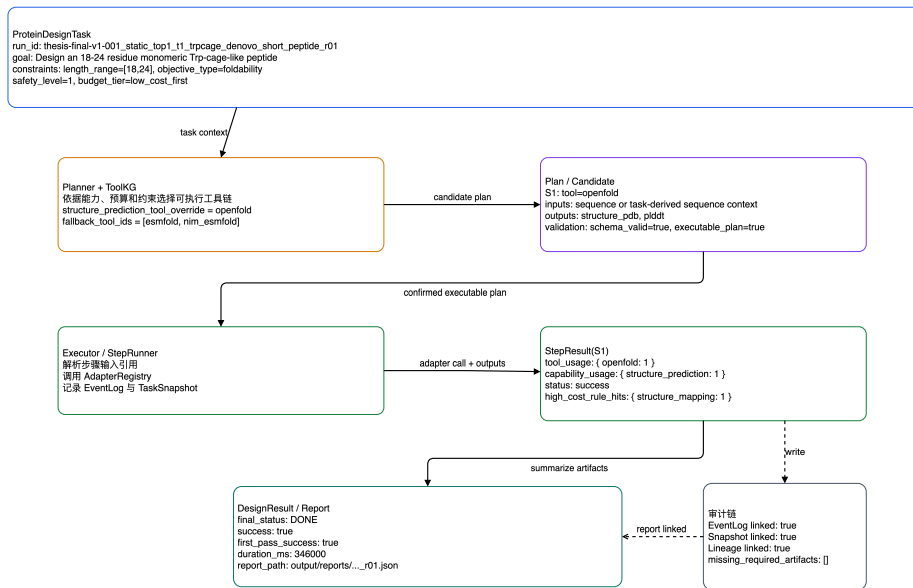


图 4-8 t1 任务实例走查

## 4.13 本章小结

本章完成了系统总体设计和 CEBRA-WP 算法定义。系统采用五层架构组织输入、规划、执行、安全汇总和资源管理，并通过 FSM 与 HITL 契约约束任务状态推进。蛋白质设计能力被组织为六阶段工作流，ProteinToolKG 则负责支持工具能力发现和 I/O 兼容校验。在算法层面，CEBRA-WP 将候选生成、硬可行性过滤、静态效用、后验目标评分、Lite belief-state、运行时重排序和恢复动作选择组合成闭环，用于高代价、部分可观测、可失败的科研工作流恢复规划。下一章将在该设计基础上说明后端 API、工作流执行、RuntimeState 更新、工具适配和前端交互等工程实现。

## 第 5 章 系统实现

本章在第四章系统设计的基础上说明工程实现方式。本文不提出新的底层蛋白质生成模型，而是把已有工具组织为可执行、可恢复、可审计的智能工作流。因此，本章主要讨论任务接入、候选计划生成、HITL 确认、工具执行、运行时恢复和结果审计等机制如何落实到代码模块和接口中。

蛋白质设计相关能力主要通过 AlphaFold/OpenFold、ESMFold、ProteinMPNN、ProtGPT2、Biopython 等工具或工具适配器接入，这些工具本身已有成熟研究基础<sup>[2,4-6,10,18]</sup>。因此，实现工作的重点在于把这些异构能力组织为可追踪、可恢复、可解释的工作流。

### 5.1 技术选型与工程结构

系统后端采用 Python 3.12、FastAPI 和 Pydantic 实现。Python 与生物信息学脚本、结构处理库和模型服务生态兼容，适合封装本地脚本和远程 REST 服务；FastAPI 提供请求路由、响应模型和自动接口文档；Pydantic 用于定义任务、计划、步骤结果、待决策对象和运行时状态等数据契约。对于蛋白质设计工作流而言，输入字段缺失、工具输出格式不一致、步骤间引用错误都可能导致后续高代价计算失败，因此系统将结构化校验前移到 API 和模型边界。

前端采用 React、TypeScript 和 Vite 构建轻量 Web 工作台。前端不保存独立业务状态，而是通过后端 API 读取任务记录、待决策对象、事件时间线和报告结果。这样可以避免前端界面与后端工作流状态分叉，使 Web 页面成为运行时状态的可视化投影。

工作流控制采用自定义 Workflow/FSM，而不是将全局控制流交给外部流程引擎。Nextflow 等科学工作流工具在可复现计算流程中具有代表性<sup>[12]</sup>，但本文系统需要在任务执行中显式处理 WAITING\_\* 状态、PendingAction、Decision、RuntimeState

和 TaskSnapshot。如果全局状态变化由外部流程引擎隐式管理，HITL 暂停、快照恢复和恢复候选审计就难以统一。因此，系统将外部工具和流程后端限定在单个 PlanStep 内，把多步编排、状态迁移和人工确认保留在自定义运行时中。

表 5-1 给出了后端核心模块与论文架构层之间的对应关系。该表说明系统实现并非简单的文件目录堆叠，而是围绕任务接入、数据契约、工作流控制、工具适配和审计恢复形成分层实现。

表 5-1 后端核心模块与论文架构层对应关系

| 实现目录          | 主要职责                                                         | 对应设计层/模块             | 正文说明重点                                            |
|---------------|--------------------------------------------------------------|----------------------|---------------------------------------------------|
| src/api/      | FastAPI 入口、任务接口、HITL 接口、前端静态入口                               | 输入交互层、任务接入模块         | API 是任务、事件、报告和人工决策的统一边界。                          |
| src/models/   | Pydantic 契约、状态枚举、任务记录                                        | 核心数据契约               | 任务、计划、步骤结果、PendingAction、RuntimeState 等对象均受结构化约束。 |
| src/agents/   | Planner、Executor、Safety、Summarizer                           | 多 Agent 协作模块         | 各 Agent 只承担明确职责，不直接越权改变全局状态。                      |
| src/workflow/ | WorkflowContext、PlanRunner、StepRunner、RuntimeEvaluator、恢复与快照 | 工作流控制层、CEBRA-WP 工程承载 | FSM、HITL、重试、恢复、runtime rerank 和快照的主要实现位置。         |
| src/adapters/ | ToolAdapter、AdapterRegistry、具体工具适配器                          | 工具执行与资源接入层           | 屏蔽本地工具、远程 REST 服务和脚本差异。                           |
| src/kg/       | ProteinToolKG JSON 与查询逻辑                                     | ProteinToolKG        | 为候选生成提供工具能力、I/O、成本和安全约束。                          |

表 5-1 (续表)

| 实现目录         | 主要职责           | 对应设计层/模块 | 正文说明重点                                  |
|--------------|----------------|----------|-----------------------------------------|
| src/storage/ | 事件日志、快照、文件产物管理 | 审计与恢复支撑  | EventLog 和 TaskSnapshot 支撑恢复、复核和实验证据提取。 |

## 5.2 任务接入与后端 API 实现

### 5.2.1 渐进式任务录入

蛋白质设计任务通常不能仅靠一句自然语言目标完整描述。用户可能输入“设计一个 30 残基的稳定短肽”，但长度范围、任务类型、结构模板、预算约束、安全限制和目标评分方式仍需要补充。为降低自由文本直接进入高代价执行的风险，系统实现了 Task Intake 链路，将任务录入拆分为草稿创建、字段补充、场景预检查和正式确认几个阶段。

后端提供 /task-intakes/schema、/task-intakes、/task-intakes/{id} 和 /task-intakes/{id}/confirm 等接口。字段注册表由后端返回，前端 Task Builder 根据字段类型、是否必填和分组信息动态渲染表单。用户补充字段后，系统进行场景门控和安全预检查，只有确认后的结构化任务规格才会进入正式任务创建路径。

在正式任务创建接口中，系统支持 goal、query 和 confirmed\_task\_spec 三种入口，但三者必须互斥。代码清单 5-1 展示了任务创建请求模型中的互斥校验。该校验保证任务来源清晰：自由文本、兼容入口和已确认结构化规格不会混杂进入同一个任务。

#### 代码清单 5-1 任务创建请求的互斥入口校验

来源：../thesis-project.dev/src/api/main.py:166

```

1 class TaskCreateRequest(BaseModel):
2     goal: Optional[str] = Field(
3         None,
4         description="蛋白质设计任务目标(自然语言)",
5     )
6     query: Optional[str] = Field(
7         None,
8         description="兼容自由文本入口；会收敛为 intake",
9     )
10    confirmed_task_spec: Optional[ConfirmedTaskSpec] = Field(

```

```

11         None,
12         description="已经确认的结构化任务输入",
13     )
14     constraints: Dict[str, Any] = Field(
15         default_factory=dict,
16         description="结构化约束",
17     )
18     metadata: Dict[str, Any] = Field(default_factory=dict)
19
20     @model_validator(mode="after")
21     def _validate_creation_mode(self) -> "TaskCreateRequest":
22         modes = [
23             self.goal is not None,
24             self.query is not None,
25             self.confirmed_task_spec is not None,
26         ]
27         if not any(modes):
28             raise ValueError(
29                 "one of goal, query, or confirmed_task_spec is required
30                 "
31             )
32         if sum(modes) > 1:
33             raise ValueError(
34                 "choose exactly one of goal, query, or
35                 confirmed_task_spec"
36             )
37         return self

```

代码清单 5-1 对应第四章中“任务输入需要结构化确认”的设计要求。该校验不直接提高蛋白质设计模型的能力，但可以避免任务入口语义不清导致后续计划生成和审计困难。

### 5.2.2 任务生命周期接口

正式任务创建后，系统通过任务生命周期接口暴露任务状态、事件和报告。GET/tasks/{task\_id} 返回任务记录，包括外部状态、内部状态、任务目标、约束、计划、设计结果、待决策对象和安全事件。外部状态用于前端展示，例如 WAITING\_PATCH\_CONFIRM; 内部状态保留执行细节, 例如 WAITING\_PATCH 或 PATCHING。这种双状态设计使 UI 可以保持简洁，同时不丢失恢复流程所需的精确状态。

GET/tasks/{task\_id}/events 返回事件时间线。该接口直接读取事件日志，而不是只依赖内存任务表。因此，只要日志文件存在，服务重启后仍可还原任务的状态迁移、步骤执行、等待进入、决策应用和等待退出等事件。GET/tasks/{task\_id}/report 返回最终报告摘要，包含序列、结构文件路径、评分、风险标记和目标评分结果；未完成任务请求报告时返回错误响应，避免把中间状态误认为最终结果。

### 5.2.3 HITL 接口

HITL 机制通过 PendingAction 和 Decision 建模。GET/pending-actions 返回待审查任务摘要，GET/pending-actions/{id} 返回候选详情，POST/pending-actions/{id}/decision 接收用户决策。候选详情中包含候选数量、默认建议、评分分解、运行时状态摘要、风险等级、成本估计和解释字段。这些信息使前端能够展示“为什么推荐该候选”，而不是只给出一个不可解释的确认按钮。

当用户提交 Decision 后，后端会验证 PendingAction 是否属于当前任务、是否仍处于待决策状态、所选候选是否存在。验证通过后，后端根据 action type 将决策应用到 plan、局部修补或重规划，并写入决策事件和等待退出事件。这一过程保证人工决策属于 workflow 状态，而不是前端临时 UI 状态。

## 5.3 前端工作台实现

前端工作台由 Dashboard、Task Builder、Task Detail 和 Event Timeline 四类页面组成。Dashboard 展示任务列表、状态摘要和工具能力 readiness；Task Builder 承担任务录入、字段补充和确认；Task Detail 聚合任务状态、候选审查、报告与结构产物入口；Event Timeline 用于查看任务生命周期事件。

前端启动时从后端注入的 bootstrap payload 获取当前视图和任务 ID，再通过 API 拉取任务、待决策对象、事件和报告。这种实现方式使前端不需要自行推导 FSM 状态，也不会在页面刷新或任务切换时产生独立状态源。

在 Task Detail 页面中，PendingReviewWorkspace 是 HITL 的主要交互区域。当任务进入 WAITING\_\* 状态时，前端展示候选方案对比、评分分解、运行时上下文和决策表单。用户提交决策后，前端仅向后端发送结构化 Decision，具体的计划应用、状态迁移、快照写入和事件记录均由后端 workflow 模块完成。

当前前端的结构展示区域主要提供结构文件路径、报告浏览和可视化入口。论文中应将其表述为“结构产物入口与展示区域”，而不扩大为完整的三维结构分析平台。



## 5.4 workflow运行时与执行引擎

系统运行时的核心执行序列如图 5-1 所示。用户通过 API 创建任务后, Workflow 创建运行时上下文, Planner 根据任务目标和 ProteinToolKG 生成候选计划, Executor 在人工确认或自动选择后执行 PlanStep。执行过程中, StepRunner 调用 ToolAdapter 解析输入、执行工具、校验输出, 并将 StepResult 写回上下文。Safety 模块在输入、步骤和输出阶段提供风险判定, Storage 模块负责事件日志、快照和产物写入。

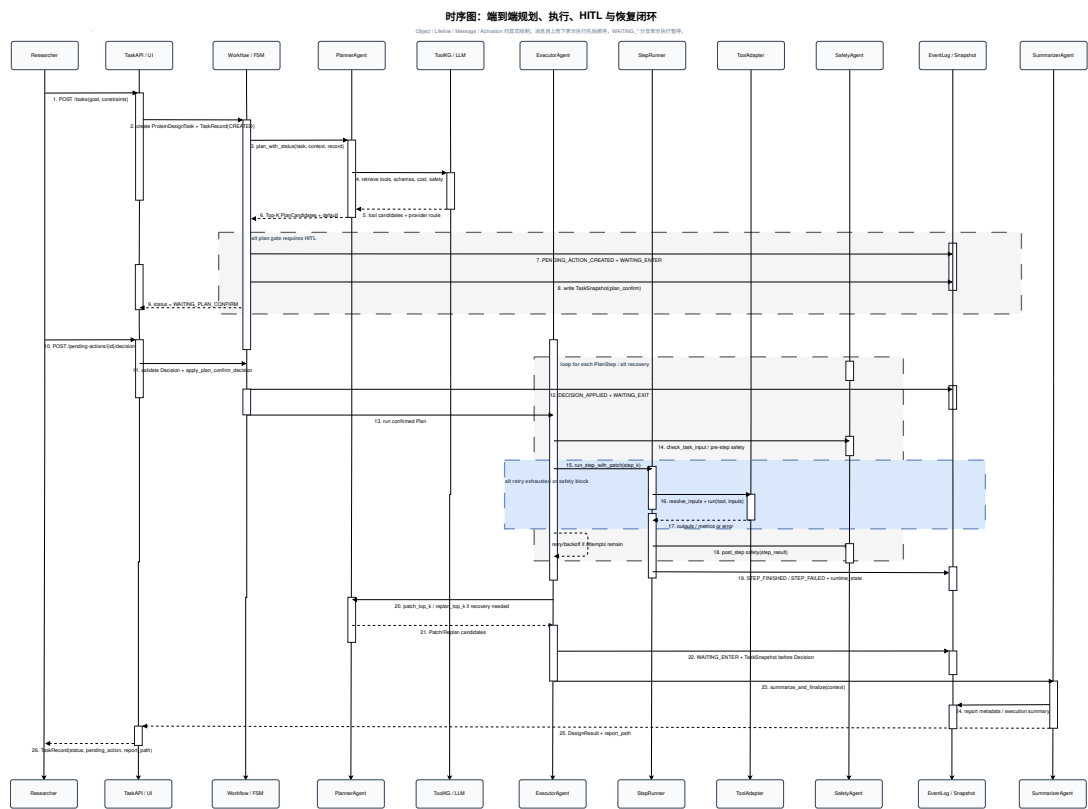


图 5-1 运行时执行序列

图 5-1 的重点在于说明 Planner 和 Executor 的边界: Planner 生成候选并查询工具知识, 不直接执行工具; Executor 只执行已确认的 PlanStep, 并通过 StepRunner 和 ToolAdapter 与外部工具交互。这一边界保证候选生成、人工确认、工具执行和审计记录可以分别追踪。

### 5.4.1 WorkflowContext

WorkflowContext 是单个任务运行期间的上下文对象, 保存任务、当前计划、步骤结果、安全事件、RuntimeState、最终结果和 PendingAction。运行时状态更新不由各模块分散修改, 而是通过上下文提供的统一入口触发。代码清单 5-2 展示了 StepResult 和 SafetyResult 写入时如何触发 RuntimeState 更新。

## 代码清单 5-2 WorkflowContext 中的 RuntimeState 更新入口

来源: ../thesis-project.dev/src/workflow/context.py:90

```
[] def add_step_result(self, result: StepResult) -> None: self.step_results[result.step_id] = result
if not runtime_policy_uses_belief_state(self.task): return self.apply_runtime_state_update(
    step_result=result, failure_context=extract_failure_context(result), )

def add_safety_event(self, event: SafetyResult) -> None: self.safety_events.append(event)
if not runtime_policy_uses_belief_state(self.task): return self.apply_runtime_state_update(safety_result=event)
```

该实现对应第四章中 Lite belief-state / 轻量信念状态的设计。系统只有在任务启用相关 runtime policy 时才更新 RuntimeState, 从而支持 static、dynamic 和 lite belief-state 等策略在同一代码基中切换。

### 5.4.2 PlanRunner 与 StepRunner

PlanRunner 负责计划级执行和 FSM 状态推进。它在计划开始执行前检查任务和计划的一致性, 进入运行状态后按顺序执行 PlanStep。若执行过程中出现可修复失败, PlanRunner 会请求局部修补候选并进入 WAITING\_PATCH\_CONFIRM; 若出现结构性失败或安全阻断, 则请求后缀重规划候选并进入 WAITING\_REPLAN\_CONFIRM。PlanRunner 可以推进合法状态迁移, 但不会绕过 PendingAction 自动应用需要人工确认的候选。

StepRunner 是单步执行的最小单元。它负责有界重试、安全检查、工具适配、输入解析、工具调用、输出校验和错误归一化。代码清单 5-3 展示了 StepRunner 的有界重试微循环。该循环只对可重试失败进行重试; 当失败不可重试或重试耗尽时, StepRunner 返回结构化 StepResult, 由上层恢复逻辑决定是否进入 patch\_local、suffix\_replan 或 stop。

## 代码清单 5-3 StepRunner 的有界重试微循环

来源: ../thesis-project.dev/src/workflow/step\_runner.py:140

```
[] def run_step(self, step: PlanStep, context: WorkflowContext) -> StepResult: """ 执行单个 PlanStep (带静态重试), 返回最终 StepResult """
last_result: StepResult | None = None
attempt_logs: list[AttemptRecord] = []
retried_any = False

for attempt_idx in range(1, self._retry_policy.max_attempts + 1):
    result = self._run_once(step, context)
    self._annotate_attempt_meta(result, attempt_idx)
    attempt_logs.append(self._build_attempt_record(result, attempt_idx))
    last_result = result

    if result.status == "success":
        return self._finalize_success(result, attempt_logs)

    failure_type = self._normalize_failure_type(result.failure_type)
    retried_any = retried_any or attempt_idx > 1
    exhausted = attempt_idx >= self._retry_policy.max_attempts
    if failure_type is
```

```

None or not is_retryable_failure(failure_type): return self._finalize_failure(result, attempt_logs,
retried_any, exhausted) if exhausted: return self._finalize_failure(result, attempt_logs, retried_any,
True)
    backoff_ms = self._retry_policy.backoff_ms_for(attempt_idx) if backoff_ms > 0: self._sleep_fn(backoff_ms
/ 1000)
    return self._finalize_failure(last_result, attempt_logs, retried_any, True)

```

该实现避免工具异常直接扩散到上层模块。无论底层问题是本地脚本失败、远程服务超时，还是输出字段缺失，PlanRunner 和 CEBRA-WP 接收到的都是统一的 failure\_type、attempt history 和错误详情。

### 5.4.3 等待状态、审计与快照

恢复闭环的关键在于进入等待状态前固化待决策对象和审计信息。代码清单 5-4 展示了 enter\_waiting\_state 的核心逻辑：状态迁移前，系统先校验等待状态和 PendingAction 的匹配关系，再写入 context、TaskRecord 和事件日志。这样，即使系统在等待人工确认期间中断，恢复后仍可以还原当前等待的决策对象、候选集合和默认建议。

#### 代码清单 5-4 进入 WAITING 状态前写入 PendingAction 与审计信息

来源：../thesis-project.dev/src/workflow/pending\_action.py:101

```

[] def enter_waiting_state( context: WorkflowContext, record: TaskRecord | None, pend-
ing_action: PendingAction, to_status: InternalStatus, *, reason: Optional[str]=None, event_logger:
EventLogger | None = None, snapshot_writer: SnapshotWriter | None = None, ) -> None: _vali-
date_waiting_transition(context, pending_action, to_status, record)
    prev_status = to_external_status(context.status) new_status = _to_external_waiting_status(to_status)
    context.pending_action = pending_action if record is not None: record.pending_action =
pending_action
    log_handler = event_logger or _default_event_logger log_handler( { "event": "PEND-
ING_ACTION_CREATED", "task_id": context.task.task_id, "pending_action_id": pending_action.pending_action_i-
"action_type": pending_action.action_type.value, "candidate_ids": [c.candidate_id for c in pend-
ing_action.candidates], "default_suggestion": pending_action.default_suggestion, "default_recommendation":
pending_action.default_recommendation, } )

```

该代码清单说明，HITL 在系统中被实现为受控运行时状态，而不是前端组件上的临时按钮。PendingAction、Decision、EventLog 和 TaskSnapshot 共同构成了可恢复、可复核的 HITL 机制。

## 5.5 CEBRA-WP 的工程落点

CEBRA-WP 在工程实现中不是一个单独的 Agent, 而是分布在候选生成、RuntimeState 更新、候选重排、恢复动作映射和候选解释几个环节。Planner 生成候选及其静态评分, WorkflowContext 维护 RuntimeState, RuntimeEvaluator 计算 runtime adjustment 和 action utility, PlanRunner 则将动作建议映射为 continue、patch\_local、suffix\_replan 或 terminal\_stop 等工作流行为。

代码清单 5-5 展示了 RuntimeEvaluator 对候选进行运行时评估和重排的入口。若策略模式禁用重排, 则系统返回静态默认候选; 若缺少 RuntimeState, 则只进行 passthrough; 当 Lite belief-state / 轻量信念状态可用时, 系统根据 RuntimeState 对候选应用 runtime adjustment, 并按 final score 重新排序。

### 代码清单 5-5 RuntimeEvaluator 候选重排

来源: ../thesis-project.dev/src/workflow/runtime\_evaluator.py:331

```
[] def evaluate_candidates( self, candidates: list[PendingActionCandidate], runtime_state: RuntimeStateSchema | Mapping[str, object] | None, ) -> RuntimeEvaluation: """ 对每个候选计算 runtime_adjustment 与 final_score, 按 final_score 降序重排。 """ if not candidates: return RuntimeEvaluation(policy_mode=self._policy_mode)
    state = _coerce_state(runtime_state) static_default = _top_static_candidate(candidates)
    if policy_disables_rerank(self._policy_mode): return RuntimeEvaluation( candidates=list(candidates), static_default_id=static_default, reranked_default_id=static_default, rerank_applied=False, policy_mode=self._policy_mode, )
    if state is None: reranked = [_apply_passthrough(c) for c in candidates] reranked.sort(key=_final_score_key, reverse=True) top = reranked[0].candidate_id if reranked else None return RuntimeEvaluation( candidates=reranked, static_default_id=static_default, reranked_default_id=top, rerank_applied=False, policy_mode=self._policy_mode, )
```

该实现对应第四章 CEBRA-WP 的运行时重排序环节。需要强调的是, RuntimeEvaluator 只在已经满足硬可行性约束的候选之间调整排序, 不绕过工具存在性、schema、I/O、安全和预算硬约束。

图 5-2 从泳道视角展示了这些模块之间的协作关系。科研人员通过 Web UI 创建任务和提交决策, Task API 负责请求与响应边界, Planner 生成候选, Executor 调度 StepRunner 和 ToolAdapter, Safety 提供风险信号, Storage 固化事件和快照。CEBRA-WP 贯穿其中: 候选生成后参与重排, 失败或风险出现时参与恢复动作选择, 并通过 PendingAction 将解释信息呈现给人工决策者。

图 5-2 说明了系统控制面和执行面的分离关系。CEBRA-WP 影响候选排序和恢复动作, 但实际状态迁移仍由 Workflow/FSM 执行; 工具调用仍由 Executor 和 ToolAdapter 完成; 人工确认仍通过 PendingAction/Decision 进入系统。

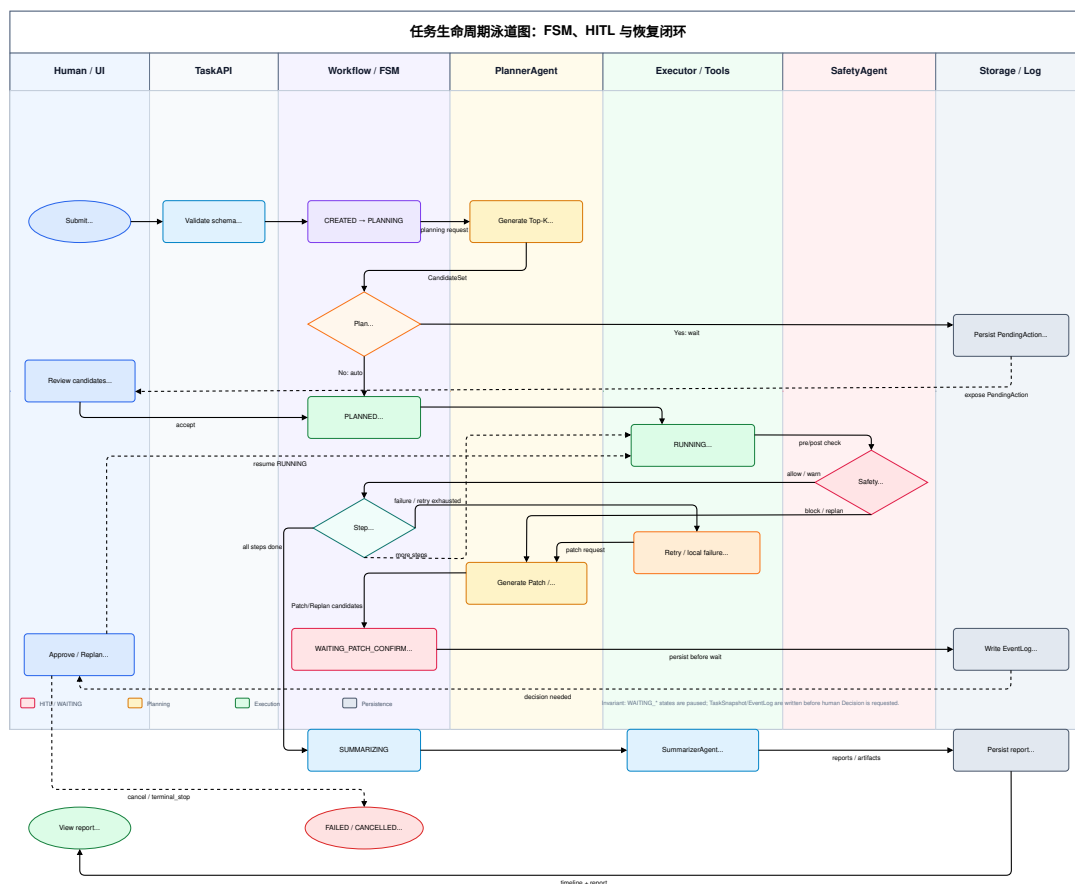


图 5-2 workflow泳道式模块协作

## 5.6 工具适配器与能力管理

上述运行时决策最终需要落到具体工具调用，因此还需说明工具适配层如何为 Executor 提供统一边界。

蛋白质设计工具链具有明显异构性：有的工具是本地 Python 脚本，有的工具依赖命令行环境，有的工具通过远程 REST 服务提供能力。为了避免 Executor 绑定具体工具实现，系统通过 BaseToolAdapter 定义统一工具接口。代码清单 5-6 展示了适配器抽象基类的核心方法。

### 代码清单 5-6 ToolAdapter 抽象接口

来源：../thesis-project.dev/src/adapters/base\_tool\_adapter.py:14

```
[] class BaseToolAdapter(ABC): """ToolAdapter 抽象基类，统一工具调用入口"""
    tool_id: str adapter_id: str | None = None
    @abstractmethod def resolve_inputs( self, step: PlanStep, context: WorkflowContext ) -> Dict[str, Any]: """ 将 PlanStep.inputs 解析为工具实际输入 """
    @abstractmethod def run_local( self, inputs: Dict[str, Any] ) -> Tuple[Dict[str, Any], Dict[str, Any]]: """ 本地执行工具并返回 (outputs, metrics) """
    def run_remote( self, inputs: Dict[str, Any], output_dir: Optional[Path] = None, ) -> Tuple[Dict[str, Any], Dict[str, Any]]: raise NotImplementedError( f"{self.__class__.__name__} does not support remote execution" )
```

该接口将输入解析、执行模式和输出指标统一到同一抽象层。对于 Executor 而言，不同工具都表现为“接收结构化输入，返回 outputs 和 metrics”的适配器。具体工具内部是调用 Biopython、远程 OpenFold 服务还是序列生成模型，对上层 workflow 保持透明。

AdapterRegistry 维护 tool\_id 到适配器实例的映射。StepRunner 执行某个 PlanStep 时，先通过 tool\_id 找到适配器，再调用 resolve\_inputs 解析常量输入和上游步骤引用。若工具未注册、上游引用缺失或输出类型不匹配，系统返回结构化失败，而不是静默跳过或继续执行。

ProteinToolKG 以 JSON 形式保存工具能力、输入输出、兼容关系、成本、安全级别和版本信息。Planner 在候选生成时查询 ProteinToolKG，根据任务所需能力组合工具链，并过滤不可用工具、预算超限工具或 I/O 闭包不成立的候选。该实现使新增工具主要通过更新 ProteinToolKG 和注册适配器完成，而不需要修改 Planner 的核心流程。

## 5.7 本章小结

本章从工程角度说明了系统实现方式。系统后端以 Python、FastAPI 和 Pydantic 为基础，前端以 React、TypeScript 和 Vite 构建工作台； workflow 控制采用自定义 Workflow/FSM，使 HITL、快照恢复和事件审计保持显式可控。

任务接入方面，系统通过 Task Intake 和互斥任务创建入口将自然语言目标逐步收敛为结构化任务。运行时方面，WorkflowContext、PlanRunner 和 StepRunner 分别承担上下文管理、计划级状态推进和步骤级工具执行。恢复方面，PendingAction、Decision、EventLog 和 TaskSnapshot 共同保证等待状态可审计、可恢复。算法落地方面，CEBRA-WP 通过 RuntimeEvaluator、RuntimeState 更新和恢复动作映射参与候选重排和运行时决策。工具接入方面，ToolAdapter 和 ProteinToolKG 将异构蛋白质设计工具封装为统一能力。

本章描述的实现模块构成第六章测试与验证的对象基础。第六章将围绕 API 合约、FSM/HITL、快照恢复、前端与 CLI 可用性，以及 retry、局部修补、后缀重规划和安全边界展开验证。

## 第 6 章 系统测试与验证

本章在第五章系统实现的基础上，验证系统功能正确性和工程可用性。与第七章的策略对比实验不同，本章关注系统是否按照第四章定义的架构、状态机、HITL 机制、工具适配机制和恢复机制运行。也就是说，本章先回答“系统能否可靠地承载实验”，第七章再讨论不同规划策略在该系统上的行为差异。

蛋白质设计 workflow 具有工具链长、单步代价高、输入输出格式差异大和失败恢复路径复杂等特点。若只检查最终任务是否完成，难以判断错误来自任务设计、工具能力、workflow 控制还是人工决策边界。因此，本章采用分层测试和证据追踪相结合的方法，将 API 合约、前端交互、命令行入口、有限状态机、HITL、快照恢复、安全边界和端到端执行纳入验证范围。科学 workflow 研究通常强调执行可靠性、数据来源追踪和失败恢复能力<sup>[27]</sup>，开放环境 Agent 评测也强调真实交互路径中的执行稳健性和状态可观测性<sup>[21]</sup>。本文据此将“是否可执行”和“是否可追溯”同时作为工程验证目标。

### 6.1 测试策略与验证目标

系统测试采用多层次、多入口的组合策略。在层次上，单元测试用于验证 Agent 职责边界、数据契约、状态迁移规则和恢复动作；集成测试用于验证 Planner、Executor、Safety、RuntimeEvaluator、Storage 等模块之间的协作；API 测试用于检查任务创建、事件读取、报告查询和决策提交等接口契约；手工验证与截图证据用于补充 Web

工作台和 CLI 的实际使用路径。

在入口上，系统覆盖 API、Web 工作台和命令行三种访问方式。API 是系统状态的统一事实来源；Web 工作台用于验证任务录入、候选审查、事件时间线和报告浏览；CLI 用于验证自动化或脚本化使用场景。三类入口读取同一任务记录、事件日志和报告产物，因此本章不仅检查单个入口是否可用，也检查同一 task\_id 在不同入口下的状态一致性。

测试用例以 TC-S01 至 TC-S13 编号，覆盖 30 余个验证点，按功能维度分为环境与能力就绪、任务录入与数据契约、计划候选生成、HITL 决策、有限状态机、快照恢复、前端与 CLI 可用性、失败恢复、安全边界和端到端执行。表 6-1 汇总了各测试用例的覆盖范围、执行结果和核心证据。

表 6-1 系统测试用例覆盖矩阵

| 用例     | 测试类别        | 覆盖验证点                | 执行结果 | 核心证据                                |
|--------|-------------|----------------------|------|-------------------------------------|
| TC-S01 | 环境与能力就绪     | SV-01、SV-27          | 通过   | EVD-API-01、EVD-API-02               |
| TC-S02 | API 合约与任务录入 | SV-02/03/04/07/25/26 | 通过   | EVD-TEST-01、EVD-API-03 至 EVD-API-08 |
| TC-S03 | 计划候选生成      | SV-08/09/10          | 通过   | EVD-TEST-02、EVD-LOG-04              |
| TC-S04 | HITL 决策一致性  | SV-10/11/12/13/15    | 通过   | EVD-TEST-02、FIG-SV-15               |
| TC-S05 | FSM 状态迁移    | SV-14/23/24          | 通过   | EVD-TEST-02、EVD-LOG-03              |
| TC-S06 | 快照持久化与恢复    | SV-14/28/29          | 通过   | EVD-TEST-02、EVD-LOG-01 至 EVD-LOG-03 |
| TC-S07 | Web 前端可用性   | SV-01、SV-30          | 通过   | FIG-SV-01 至 FIG-SV-18、EVD-TEST-01   |



表 6-1（续表）

| 用例     | 测试类别       | 覆盖验证点                   | 执行结果 | 核心证据                                |
|--------|------------|-------------------------|------|-------------------------------------|
| TC-S08 | CLI 可用性    | SV-02/03/25/26/30       | 部分通过 | EVD-CLI-01 至 EVD-CLI-04、EVD-TEST-04 |
| TC-S09 | 端到端任务流程    | SV-16/25/26             | 通过   | EVD-EXP-01、EVD-LOG-05               |
| TC-S10 | 异常输入与安全边界  | SV-04/05/06/11/13/21/24 | 通过   | EVD-TEST-03                         |
| TC-S11 | 工具链执行与 I/O | SV-16/17                | 通过   | EVD-TEST-03、EVD-EXP-01              |
| TC-S12 | 失败恢复流程     | SV-18/19/20/22          | 通过   | EVD-TEST-03、EVD-LOG-01 至 EVD-LOG-03 |
| TC-S13 | 恢复止损与审计链路  | SV-21/23/25             | 通过   | EVD-TEST-02/03、EVD-LOG-03/04        |

由表 6-1 可见，13 个测试用例中 12 个通过，1 个部分通过。部分通过项为 CLI 可用性验证：intake schema 和 task show 等核心命令可用，timeline show 与 report show 当前仍输出 usage 提示，因此在终稿中作为实现限制说明。

各测试用例的设计目的、前置条件、触发方式、预期结果和证据边界见附录 A。正文后续小节只展开与系统正确性论证直接相关的关键路径，避免将测试文档式细节全部放入正文。

本章使用的证据材料按编号前缀组织，如表 6-2 所示。正文仅保留必要证据编号，完整文件路径由系统验证材料索引维护。FIG-SV-01 至 FIG-SV-18 属于前端验证截图证据，本章不将其作为正式“图 6-x”编号插图，以避免正文图号与证据截图编号混用。

表 6-2 系统验证证据类型索引

| 证据前缀     | 证据类型                            | 当前数量/范围 | 主要支撑内容                                                    | 典型章节位置      |
|----------|---------------------------------|---------|-----------------------------------------------------------|-------------|
| EVD-API  | API 响应<br>JSON                  | 8 项     | 健康检查、能力 readiness、任务详情、事件、报告和 pending action 接口           | 6.2、6.3、6.4 |
| EVD-TEST | pytest 执行日志                     | 4 组     | API、Web smoke、FSM/HITL/快照、安全恢复和 CLI 测试结果                  | 6.1 至 6.8   |
| EVD-CLI  | CLI 输出日志                        | 4 项     | intake schema 与 task show 可用，<br>timeline/report 子命令为当前限制 | 6.5         |
| FIG-SV   | 前端截图                            | 18 张    | Dashboard、Task Builder、Task Detail、Timeline 页面可用性         | 6.5 或附录     |
| EVD-LOG  | EventLog / Snapshot / Report 样本 | 8 组     | 状态迁移、恢复闭环、terminal_stop、等待态快照与报告产物                        | 6.4 至 6.8   |
| EVD-EXP  | 实验矩阵与端到端运行聚合                    | 4 项     | smoke/clean run、工具链执行和端到端流程证据                             | 6.7、第七章     |

表 6-2 用于明确不同证据承担的论证功能。API 响应支撑接口契约, pytest 日志支撑自动化验证结果, CLI 输出和前端截图支撑交互入口可用性, EventLog 与 Snapshot 支撑运行时状态可追溯性, 实验聚合结果则说明系统能够承载批量端到端任务。

## 6.2 API 服务与任务录入验证

系统基础可用性通过健康检查和能力 readiness 接口验证。健康检查接口返回 `status=ok`、任务数量、ProteinToolKG 中的能力数量, 以及日志、快照和输出目录等路径信息。能力 readiness 接口返回 15 条蛋白质设计工具或工具能力的运行状态, 其中可直接调用的能力标记为 `ready`, 远程服务不可达但不阻断系统运行的能力标记为 `degraded`, 当前环境缺失的外部工具标记为 `unavailable`。每条 `degraded` 或 `unavailable` 记录均包含错误类别和建议恢复方式。这表明, 系统没有把工具可用性写成静态假设, 而是在运行时向 Planner 和用户暴露当前环境的真实能力边界。

任务录入验证对应 TC-S02。系统通过 Task Intake 链路将自然语言目标转化为结构化任务规格, 主要包括字段 `schema` 获取、草稿创建、字段补充、场景预检查和确认创建任务几个步骤。`/task-intakes/schema` 返回完整字段注册表, 用于支撑前端动态表单; 自由文本可以创建 `intake` 草稿; 缺失必要字段时 `confirm` 请求被拒绝; `goal`、`query` 和 `confirmed_task_spec` 三种创建模式同时出现时被明确拒绝。这些验证对应第五章代码清单 5-1 中的互斥入口校验, 说明实现层与测试层在任务边界上保持一致。

任务创建后的生命周期接口也被纳入验证范围。`/tasks/{task_id}` 返回任务目标、约束、计划、运行状态和报告路径等信息; `/tasks/{task_id}/events` 返回任务事件时间线; `/tasks/{task_id}/report` 在任务完成后返回设计结果摘要, 在任务未完成时返回错误响应。对真实实验任务的 `events` 查询能够直接从磁盘日志读取 25KB 事件链, 说明事件接口不依赖当前进程内的内存任务表, 具备服务重启后追溯任务过程的基础。

## 6.3 计划候选与 HITL 验证

计划候选生成由 TC-S03 覆盖。测试确认 PlannerAgent 生成的 PlanCandidate 包含 `candidate_id`、`score_breakdown`、`risk_level`、`cost_estimate`、`explanation` 和 `source_refs` 等必需字段。候选计划不仅是可执行步骤列表, 还包含可用于比较、解释和审计的评分与风险信息。测试同时确认 Planner 只负责生成候选, 不直接执行工具, 也不越权改变任务状态; 这种边界与第四章的多 Agent 职责划分一致。

当候选计划置信度不足, 或恢复动作需要人工确认时, 系统通过 PendingAction

和 Decision 进入 HITL 流程。TC-S04 验证了 PendingAction 的创建、决策提交和异常边界。进入 WAITING\_PLAN\_CONFIRM、WAITING\_PATCH\_CONFIRM 或 WAITING\_REPLAN\_CONFIRM 状态前，系统必须创建对应的 PendingAction；提交 accept 决策时必须包含合法的 selected\_candidate\_id；对已处理的 PendingAction 重复提交决策会返回冲突；将 Decision 提交到不属于该任务的 PendingAction 会被拒绝。

这些验证表明，HITL 在系统中不是前端界面上的自由按钮，而是受状态机约束的运行时契约。等待态下 Executor 不会继续调用工具；决策生效后，系统写入 DECISION\_APPLIED 和 WAITING\_EXIT 等事件，再按有限状态机规则迁移到后续状态。EVD-LOG-08 中记录的 PENDING\_ACTION\_CREATED -> WAITING\_ENTER -> DECISION\_SUBMITTED -> DECISION\_APPLIED -> WAITING\_EXIT 事件链，展示了人工决策从创建到应用的完整审计路径。

## 6.4 状态机、快照与恢复基础验证

有限状态机验证对应 TC-S05。系统允许的状态迁移路径包括从 CREATED 到 PLANNING，再到 PLANNED 或等待人工确认状态；计划执行后进入 RUNNING，并根据执行结果进入等待局部修补、等待后缀重规划、总结或终态。测试覆盖了合法迁移和关键非法迁移，确认不在规则集中的状态转换会被拒绝。DONE、FAILED 和 CANCELLED 作为终态，一旦进入便不可再被普通状态更新覆盖。

快照验证对应 TC-S06。系统要求进入任意 WAITING\_\* 状态前完成 PendingAction 写入、事件日志记录和 TaskSnapshot 保存。该顺序保证了系统即使在等待人工确认期间中断，也可以在恢复后还原 pending action、候选集合、已完成步骤、计划版本和运行时状态。恢复到等待态后，系统不会自动推进执行，而是继续等待人工 Decision，这一点对于保护 HITL 决策边界尤为关键。

快照验证还确认了运行时状态与计划语义字段的隔离。Lite belief-state / 轻量信念状态中的 p\_success、budget\_pressure、recovery\_margin 等字段保存在 snapshot artifacts 的 RuntimeState 中，而 Plan 本身仍保持步骤、输入输出和约束定义。这种隔离避免了运行时估计污染计划语义，也使不同策略配置可以在同一执行框架下比较。

## 6.5 前端与 CLI 可用性验证

系统的工程可用性不仅取决于后端接口，也取决于用户实际接触的交互入口。TC-S07 验证 Web 工作台，TC-S08 验证 CLI。表 6-3 汇总了两类入口的覆盖范围、证据和限制。

表 6-3 前端与 CLI 可用性证据汇总

| 入口             | 覆盖范围                                       | 证据编号                                        | 结论   | 限制                                        |
|----------------|--------------------------------------------|---------------------------------------------|------|-------------------------------------------|
| Web Dashboard  | 任务列表、状态摘要、能力提示                             | FIG-SV-01、FIG-SV-02、EVD-TEST-01             | 通过   | 以截图和 smoke test 为主，复杂浏览器兼容性仍需另行验证。        |
| Task Builder   | 字段注册表、草稿补充、安全预检查、任务确认                      | FIG-SV-03 至 FIG-SV-11、EVD-API-03            | 通过   | 字段语义由 API schema 支撑，截图只作为交互证据。            |
| Task Detail    | 任务状态、运行上下文、候选决策、报告与结构区域                    | FIG-SV-12 至 FIG-SV-17、EVD-API-04、EVD-API-06 | 通过   | 结构区域应表述为产物入口与展示区域，不扩大为完整三维结构分析平台。         |
| Event Timeline | 状态迁移、步骤完成、等待进入、决策应用、等待退出                   | FIG-SV-18、EVD-API-05                        | 通过   | 当前以单任务样本展示全生命周期。                          |
| CLI            | intake schema、task show、timeline/report 命令 | EVD-CLI-01 至 EVD-CLI-04、EVD-TEST-04         | 部分通过 | timeline 和 report 子命令当前仅输出 usage，需作为限制说明。 |

Web 工作台的验证覆盖四类页面。Dashboard 展示任务列表、状态摘要和能力提示；Task Builder 覆盖从自由文本输入到字段补充、安全预检查和任务确认的流程；Task Detail 展示任务状态、运行上下文、候选决策区域、报告与结构产物入口；Event Timeline 展示状态迁移、步骤完成、等待进入、决策应用和等待退出事件。前端 smoke 测试和截图证据表明，用户可以通过 Web 入口完成任务查看、候选审查和事件追踪。

CLI 验证显示，`intake schema --json` 可以输出完整字段注册表，`task show` 可以展示任务 ID、状态、15 条能力 readiness 和恢复建议。CLI 自动化测试覆盖 16

个用例并全部通过。但 timeline show 和 report show 当前尚未提供完整展示能力，因此 TC-S08 标记为部分通过。该限制不影响 Web 和 API 对事件与报告的访问，但在终稿中需要如实说明。

### 6.6 失败恢复与安全边界验证

蛋白质设计 workflow 中，工具失败、输出缺失、远程服务不可达和安全风险都可能导致任务中断。本文系统将失败处理拆分为有界重试、局部修补、后缀重规划和终止止损几个层次。TC-S12 和 TC-S13 分别验证恢复流程和止损审计，TC-S10 验证安全边界。表 6-4 汇总了恢复与安全相关的主要证据。

表 6-4 恢复与安全边界验证证据表

| 验证主题     | 覆盖用例          | 关键行为                                                        | 主要证据                               | 结论边界                                         |
|----------|---------------|-------------------------------------------------------------|------------------------------------|----------------------------------------------|
| 有界重试     | TC-S12        | 可重试失败进入有限重试，耗尽后交给恢复逻辑                                       | EVD-TEST-03、EVD-LOG-01             | 证明 retry 机制可达，不等同于所有工具失败都可恢复。                |
| 局部修补     | TC-S12        | retry exhausted 后生成 patch_local 候选并进入 WAITING_PATCH_CONFIRM | EVD-TEST-03、EVD-LOG-01、EVD-LOG-02  | 证明局部修补路径可执行，具体成功依赖失败类型。                      |
| 后缀重规划/止损 | TC-S12、TC-S13 | 结构性失败或 terminal_stop 通过 FSM 进入重规划确认或 FAILED                 | EVD-TEST-02、EVD-TEST-03、EVD-LOG-03 | 证明 stop 不绕过 FSM/HITL。                        |
| 安全 block | TC-S10、TC-S13 | forbidden motif 在 pre-step 阶段阻断工具调用                         | EVD-TEST-03                        | 矩阵实验未充分触发 safety block，结论主要来自 focused tests。 |

表 6-4 (续表)

| 验证主题    | 覆盖用例   | 关键行为                | 主要证据                  | 结论边界                   |
|---------|--------|---------------------|-----------------------|------------------------|
| 安全 warn | TC-S10 | warn 放行但记录风险标记和安全事件 | EVD-TEST-03、FIG-SV-10 | 证明风险可记录，不宣称自动生物安全判定完备。 |

有界重试验证确认，StepRunner 遇到可重试失败时会在限定次数内重试；若重试成功， workflow 继续推进；若重试耗尽，则返回结构化失败结果，由上层恢复逻辑决定是否进入局部修补或后缀重规划。局部修补验证通过确定性触发 `retry exhausted`，随后生成 `patch_local` 候选并进入等待人工确认状态。相关日志显示，任务完成后可提取 `patch_event_count=1`、`first_pass_success=False`、`replan_event_count=0` 等恢复指标，说明恢复事件能够被后续实验统计利用。

当局部修补不足以恢复任务时，系统进入后缀重规划或止损路径。后缀重规划优先保留已完成前缀，仅替换失败后的后续步骤；`terminal_stop` 则作为终止型候选进入 HITL 确认。测试确认，接受 `terminal_stop` 后任务进入 FAILED 终态，并在事件日志中记录等待进入、决策应用和等待退出链路。该结果表明，止损路径属于可审计的工作流决策，而不是绕过状态机的异常退出。

安全边界验证覆盖输入、步骤和输出阶段的风险处理。focused tests 确认 forbidden motif 在 pre-step 阶段可以触发 block，并阻止工具调用；无 forbidden motif 时不会误阻断；warn 场景下系统允许继续执行，但会记录风险标记和安全事件。批量实验中的安全探测任务未充分触发 safety block，因此安全机制的可达性主要由确定性测试支撑，不能扩大为对所有生物安全风险的自动判定能力。

6.7 端到端流程与工具链验证

端到端流程验证由 TC-S09 和 TC-S11 覆盖。TC-S09 关注从任务输入到 Design-Result 的完整成功路径，TC-S11 关注工具链执行和 I/O 边界。t8 四组 smoke run 和 t9 四组 clean run 共覆盖 20 次运行，涉及 denovo、sequence evaluation、patchable 和 safety 等任务类型，全部以 DONE 终态完成；对应证据为 EVD-EXP-01、EVD-LOG-05 和第七章表 7-8 所列实验产物。聚合结果显示，这些运行的 success rate、schema valid rate 和 executable plan rate 均为 1.0。

每个 DONE 任务至少包含一个 StepResult，报告接口可以返回 scores、risk flags、report path 或 structure artifact 等结果字段。未完成任务请求 report 时返回 404，这一边界避免了系统将中间状态误呈现为最终结果。工具链执行方面，实验运行中的 openfold、protgpt2、biopython\_qc 等工具调用均以 success 状态完成，未出现 dummy

输入或 provider payload validation error。针对候选 schema 不匹配和上游 I/O 引用错误的确定性测试也确认，系统会在执行前结构化拒绝无效候选，而不是让错误在工具调用阶段静默扩散。

端到端验证说明，前述 API、状态机、HITL、快照、恢复和工具适配机制能够在完整任务中协同工作。第七章的 84-run 策略对比实验正是在这一工程验证基础上展开，因此实验结果中的差异可以优先归因于策略配置，而不是基本系统功能失效。

## 6.8 本章小结

本章围绕系统功能正确性和工程可用性，整理了 13 个测试用例、30 余个验证点和六类证据材料。结果表明，系统在 API 服务、任务录入、候选生成、HITL 决策、有限状态机、快照恢复、前端入口、CLI 核心命令、失败恢复、安全边界和端到端执行方面均具备可追溯证据。

从执行结果看，TC-S01 至 TC-S07、TC-S09 至 TC-S13 均通过，TC-S08 部分通过。主要限制包括：CLI 的 timeline/report 子命令尚未完整实现；前端截图能够证明关键页面可用，但不等同于全面浏览器兼容性测试；安全 block 路径主要由确定性 focused tests 支撑，批量实验中的安全任务未充分触发阻断。上述限制不会否定系统作为实验平台的基本可用性，但需要在论文结论中保留边界。

总体来看，在本文测试范围内，系统可以承载任务创建、运行时控制、人工决策、工具执行、恢复处理和审计追踪。该结果为第七章的 CEBRA-WP 策略对比实验提供了工程基础：只有系统执行链路和证据链路可用，后续关于成功率、首次成功率、高代价调用次数、恢复事件和运行时间的比较才具有解释意义。

## 第 7 章 实验与结果分析

第六章已验证系统在任务创建、状态控制、HITL、快照恢复、工具执行和审计追踪等方面具备可运行的工程基础。本章进一步分析 CEBRA-WP 相关策略在批量蛋白质设计工作流中的行为差异。实验目标限定为工作流层的规划、运行时观测、恢复控制和成本控制，不涉及候选蛋白的湿实验功能验证。

本章围绕四个研究问题展开：

- RQ-A1：CEBRA-WP 的运行时状态、候选重排和动作效用链路是否可执行、可追踪。
- RQ-A2：相比静态单链和固定阈值门控，动态观测是否改变系统行为。
- RQ-A3：Lite belief-state / 轻量信念状态相比 dynamic\_no\_belief\_state 是否提供额外机制信息。
- RQ-A4：失败和恢复案例是否能由事件日志、快照和候选 metadata 还原。



# 7.1 实验设计

实验采用四组内部消融设计：static\_top1、fixed\_threshold\_gate、dynamic\_no\_belief\_state 和 lite\_belief\_state。四组策略按照运行时介入深度递进：静态 Top-1 只选择静态最高分候选；固定阈值门控组保留阈值门控和局部修补触发；动态观测组保留恢复链路但不启用显式 Lite belief-state / 轻量信念状态；lite\_belief\_state 组启用运行时状态、runtime adjustment 和 action utility。该设计借鉴了 LLM Agent 中“推理-行动”、候选搜索和失败反馈机制的思想<sup>[8,19,20]</sup>，但实验对象限定为本文系统中的 workflow 规划与恢复控制。

如图 7-1 所示，实验矩阵由任务集、策略组、评价指标和证据产物四部分组成。任务集提供不同难度、预算和失败压力；策略组用于隔离静态选择、固定阈值门控、动态恢复和 Lite belief-state / 轻量信念状态的影响；评价指标覆盖成功率、首次成功、高代价调用、恢复事件和机制可观测性；证据产物由 run config、event log、snapshot、report 和聚合 CSV 组成。

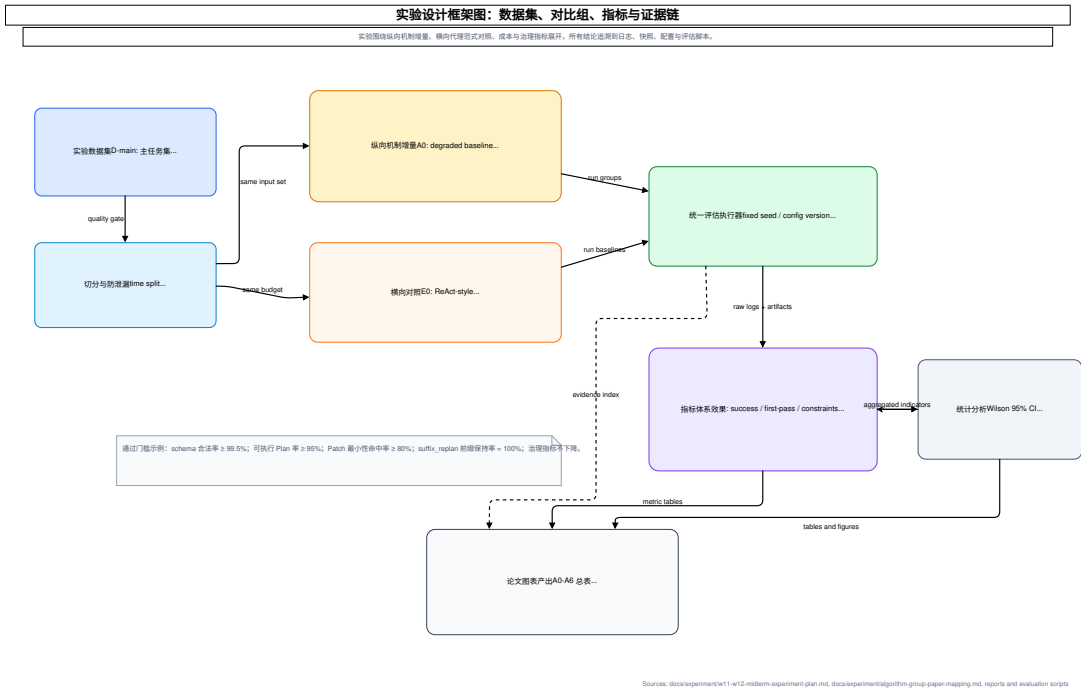


图 7-1 实验设计框架

图 7-1 的作用是限定本章统计结果的来源。后续表 7-1 至表 7-8 均来自 thesis-final-v1-001 实验矩阵及其配套日志、快照和聚合产物；对应内部证据入口为 ../thesis-project.dev/docs/experiment/thesis-final-v1-results.md 和 ../thesis-project.dev/output/experiment/thesis-final-matrix/thesis-final-v1-001/。

表 7-1 实验矩阵配置表

| 项目               | 配置                                                                                     |
|------------------|----------------------------------------------------------------------------------------|
| run_id           | thesis-final-v1-001                                                                    |
| freeze_id        | issue209-baseline-freeze-20260326                                                      |
| planner_provider | deepseek-v4-pro                                                                        |
| 任务覆盖             | 12 个 task_keys, 覆盖 T1 至 T8 共 8 类场景                                                     |
| 策略组              | static_top1 / fixed_threshold_gate /<br>dynamic_no_belief_state /<br>lite_belief_state |
| repeats          | low_cost_first=2, standard=2,<br>high_cost_sensitive=1                                 |
| 总 runs           | 84 runs, 每组 21 runs                                                                    |
| 终态结果             | 81 DONE, 3 FAILED                                                                      |
| 执行时间             | 约 6 小时                                                                                 |
| 产物路径             | ../thesis-project.dev/output/experiment/thesis-final-matrix/thesis-final-v1-001/       |

任务集覆盖 8 类场景：简单 de novo 设计、序列评估、多约束稳定性优化、高代价结构预测、可修复参数失败、远程服务降级、结构性重规划和安全探测。涉及的公开结构包括 Trp-cage、Villin HP35、GB1、Ubiquitin、Top7 和 de novo oligomer。任务定义与分层依据来自 ../thesis-project.dev/docs/experiment/final-thesis-experiment-design.md 和 ../thesis-project.dev/docs/experiment/thesis-final-v1-results.md。该任务集使实验同时包含低成本成功路径、中等压力路径和高代价结构预测路径。

## 7.2 指标定义

本章使用五类指标。任务完成指标包括 success\_rate、first\_pass\_success\_rate、schema\_valid\_rate 和 executable\_plan\_rate。恢复行为指标包括 patch\_events\_mean、replan\_events\_mean 和 suffix\_replan\_events\_mean。成本指标包括 high\_cost\_call\_mean、high\_cost\_call\_total 和 duration\_ms\_mean，其中高代价调用在本实验中主要对应结构预测与结构精修。机制可观测性指标包括 runtime\_state\_observable\_rate、runtime\_state\_summary、budget\_pressure\_source 和 action\_utility\_source。

增量指标使用相邻策略组的 paired delta。

局部修补/重规划事件来自 event log，表示实际执行过的恢复动作；action utility 来自候选 metadata，表示算法对 continue、patch\_local、suffix\_replan 和 stop 等动作的评分。两者含义不同，后文表格分别列示。

### 7.3 四组消融主结果

表 7-2 汇总四组策略的核心指标。每组 21 runs，static\_top1 全部 DONE，其余三组各 1 个 FAILED。

表 7-2 四组消融主实验结果

| 指标                            | static<br>_top1 | fixed<br>_threshold<br>_gate | dynamic<br>_no_belief<br>_state | lite<br>_belief<br>_state |
|-------------------------------|-----------------|------------------------------|---------------------------------|---------------------------|
| runs                          | 21              | 21                           | 21                              | 21                        |
| DONE                          | 21              | 20                           | 20                              | 20                        |
| FAILED                        | 0               | 1                            | 1                               | 1                         |
| success_rate                  | 1.0000          | 0.9524                       | 0.9524                          | 0.9524                    |
| first_pass_success_rate       | 1.0000          | 0.9048                       | 0.9524                          | 0.9524                    |
| schema_valid_rate             | 1.0000          | 1.0000                       | 0.9524                          | 1.0000                    |
| executable_plan_rate          | 1.0000          | 0.9524                       | 1.0000                          | 1.0000                    |
| waiting_chain_complete_rate   | 1.0000          | 1.0000                       | 1.0000                          | 1.0000                    |
| failure_traceable_rate        | 1.0000          | 1.0000                       | 1.0000                          | 1.0000                    |
| runtime_state_observable_rate | 0.0000          | 0.0000                       | 0.0000                          | 1.0000                    |
| patch_events_mean             | 0.0000          | 0.2857                       | 0.0000                          | 0.0000                    |
| replan_events_mean            | 0.0000          | 0.0000                       | 0.0000                          | 0.0000                    |
| suffix_replan_events_mean     | 0.0000          | 0.0000                       | 0.0000                          | 0.0000                    |
| high_cost_call_mean           | 1.0000          | 1.3333                       | 0.9524                          | 0.9524                    |

表 7-2 (续表)

| 指标                          | static<br>_top1 | fixed<br>_threshold<br>_gate | dynamic<br>_no_belief<br>_state | lite<br>_belief<br>_state |
|-----------------------------|-----------------|------------------------------|---------------------------------|---------------------------|
| high_cost_call<br>_total    | 21              | 28                           | 20                              | 20                        |
| duration_ms_me<br>an        | 241,905         | 300,238                      | 226,571                         | 272,095                   |
| action_continu<br>e_mean    | 1.0952          | 1.5238                       | 0.9524                          | 0.9524                    |
| action_patch_l<br>ocal_mean | 0.0000          | 0.2857                       | 0.0000                          | 0.0000                    |

表 7-2 显示出三个主要现象。static\_top1 在该矩阵中成功率最高, 提示任务集中的多数初始候选已经具备较强可执行性。fixed\_threshold\_gate 是唯一触发真实局部修补的组, 平均局部修补次数为 0.2857, 同时高代价调用总数升至 28。lite\_belief\_state 是唯一 runtime\_state\_observable\_rate 为 1.0000 的组, 说明其 Lite belief-state / 轻量信念状态链路在所有 run 中均产生可追踪输出。

## 7.4 分层结果与机制增量

表 7-3 按任务难度和预算层级汇总成功率。3 个 FAILED run 全部位于 medium/standard 层。

表 7-3 按任务难度与预算分层的成功率

| 分层                  | runs | DONE | FAILED | success_rate |
|---------------------|------|------|--------|--------------|
| easy                | 32   | 32   | 0      | 1.0000       |
| medium              | 40   | 37   | 3      | 0.9250       |
| hard                | 12   | 12   | 0      | 1.0000       |
| low_cost_first      | 32   | 32   | 0      | 1.0000       |
| standard            | 40   | 37   | 3      | 0.9250       |
| high_cost_sensitive | 12   | 12   | 0      | 1.0000       |

分层结果说明, 失败主要来自中等难度、标准预算任务。具体任务中, t2\_ubiquitin\_sequence\_ev 贡献 2 个 FAILED, t3\_gb1\_stability\_optimization 贡献 1 个 FAILED。hard 层样本量较小, 该行仅作为本矩阵中的观察事实。

表 7-4 给出相邻策略组的 paired delta。delta 为后一个策略组减前一个策略组。

表 7-4 机制增量配对对比

| 对比                | 指标                      | delta   | 样本数 | 结果含义                            |
|-------------------|-------------------------|---------|-----|---------------------------------|
| static<br>→fixed  | success_rate            | -0.0476 | 21  | fixed 组多 1 个失败。                 |
| static<br>→fixed  | first_pass_success_rate | -0.0952 | 21  | fixed 组首次成功率下降。                 |
| static<br>→fixed  | patch_event_count       | +0.2857 | 21  | fixed_threshold_gate 组触发真实局部修补。 |
| static<br>→fixed  | high_cost_call_count    | +0.3333 | 21  | fixed 组高代价调用增加。                 |
| fixed<br>→dynamic | high_cost_call_count    | -0.3810 | 21  | dynamic 组高代价调用低于 fixed。         |
| fixed<br>→dynamic | duration_ms             | -73,667 | 21  | dynamic 组平均耗时低于 fixed。          |
| dynamic<br>→lite  | schema_valid_rate       | +0.0476 | 21  | lite 组 schema valid 恢复到 1.0000。 |
| dynamic<br>→lite  | high_cost_call_count    | 0.0000  | 21  | 两组高代价调用均值相同。                    |
| dynamic<br>→lite  | duration_ms             | +45,524 | 21  | lite 组平均耗时高于 dynamic。           |

表 7-4 表明, fixed\_threshold\_gate 引入了可观测的局部修补行为, 同时带来额外高代价调用。dynamic\_no\_belief\_state 相比 fixed\_threshold\_gate 降低了高代价调用和耗时。lite\_belief\_state 相比 dynamic\_no\_belief\_state 的主要差异, 则体现在 schema valid、RuntimeState 和 action utility 等机制信息上。

## 7.5 成本与恢复行为分析

表 7-5 单独列出高代价调用和运行时间。fixed\_threshold\_gate 的高代价调用总数为 28, 其中结构映射 27 次, 结构精修 1 次; dynamic\_no\_belief\_state 和 lite\_belief\_state 均为 20 次。

表 7-5 高代价调用与运行时间对比

| 组                       | high_cost_total | high_cost_mean | 结构映射 | 结构精修 | 平均耗时       |
|-------------------------|-----------------|----------------|------|------|------------|
| static_top1             | 21              | 1.0000         | 21   | 0    | 241,905 ms |
| fixed_threshold_gate    | 28              | 1.3333         | 27   | 1    | 300,238 ms |
| dynamic_no_belief_state | 20              | 0.9524         | 20   | 0    | 226,571 ms |
| lite_belief_state       | 20              | 0.9524         | 20   | 0    | 272,095 ms |

由表 7-5 可计算得出, 与 fixed\_threshold\_gate 相比, dynamic\_no\_belief\_state 和 lite\_belief\_state 的高代价调用总数从 28 降至 20, 降幅为 28.6%。与 static\_top1 相比, 两组高代价调用总数从 21 降至 20, 差异来自各自 1 个失败 run 在高代价结构预测前终止。运行时间上, dynamic\_no\_belief\_state 最短, fixed\_threshold\_gate 最长, lite\_belief\_state 介于二者之间。

恢复事件方面, 四组均未产生真实重规划或后缀重规划; 真实局部修补仅出现在 fixed\_threshold\_gate 组, 总数 6 次, 均为 tool-level patch。该结果说明, 本次矩阵对局部修补机制产生了真实压力, 但对重规划机制的矩阵级触发不足。第六章中的 focused tests 已覆盖后缀重规划和 terminal\_stop 的可达性, 因此本章的批量实验结论以局部修补和高代价调用为主。

## 7.6 Lite belief-state 机制可观测性

表 7-6 对比 Lite belief-state 与其余三组的机制可观测性。该表对应 RQ-A1 和 RQ-A3。

表 7-6 Lite belief-state 机制可观测性对比

| 特征                            | static_top1 /<br>fixed_threshold_gate /<br>dynamic_no_belief_state | lite_belief_state             |
|-------------------------------|--------------------------------------------------------------------|-------------------------------|
| runtime_state_summary         | 无有效运行时状态摘要                                                         | 21/21 runs 产生<br>RuntimeState |
| runtime_state_observable_rate | 0.0000                                                             | 1.0000                        |

表 7-6 (续表)

| 特征                     | static_top1 /<br>fixed_threshold_gate /<br>dynamic_no_belief_state | lite_belief_state                                                                                           |
|------------------------|--------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| belief-state 核心字段      | 未观测                                                                | p_success、<br>p_structural_failure、<br>recovery_margin、<br>expected_remaining_cost、<br>evidence_sufficiency |
| budget_pressure source | default                                                            | observed                                                                                                    |
| action_utility_source  | missing                                                            | computed                                                                                                    |
| action_utilities       | 空对象或不可用                                                            | continue、patch_local、<br>suffix_replan、stop 均<br>有 utility                                                  |
| high_cost_call_mean    | static=1.0000,<br>fixed=1.3333,<br>dynamic=0.9524                  | 0.9524                                                                                                      |

Lite belief-state / 轻量信念状态的核心证据是 21/21 runs 均产生有效 RuntimeState, 且 budget pressure 来自运行时观测而非默认回退。该结果说明, CEBRA-WP 的运行时状态更新、预算压力估计和动作效用计算均已在矩阵实验中执行。与 dynamic\_no\_belief\_state 相比, lite\_belief\_state 的 success\_rate 和 high\_cost\_call\_mean 相同, 其增量主要体现为完整的 RuntimeState 和动作效用记录。

## 7.7 典型失败案例

如图 7-2 所示, fixed\_threshold\_gate 和 lite\_belief\_state 的恢复控制路径不同。fixed\_threshold\_gate 主要体现为运行时门控后的局部修补; lite\_belief\_state 则在候选评估阶段产生 RuntimeState、runtime adjustment 和 action utility, 并据此影响候选排序。

图 7-2 用于解释表 7-7 中的失败案例。fixed\_threshold\_gate 的失败体现为局部修补循环耗尽; lite\_belief\_state 的失败体现为运行时状态可观测但仍未打破循环; dynamic\_no\_belief\_state 的失败体现为候选 I/O 闭包校验失败。

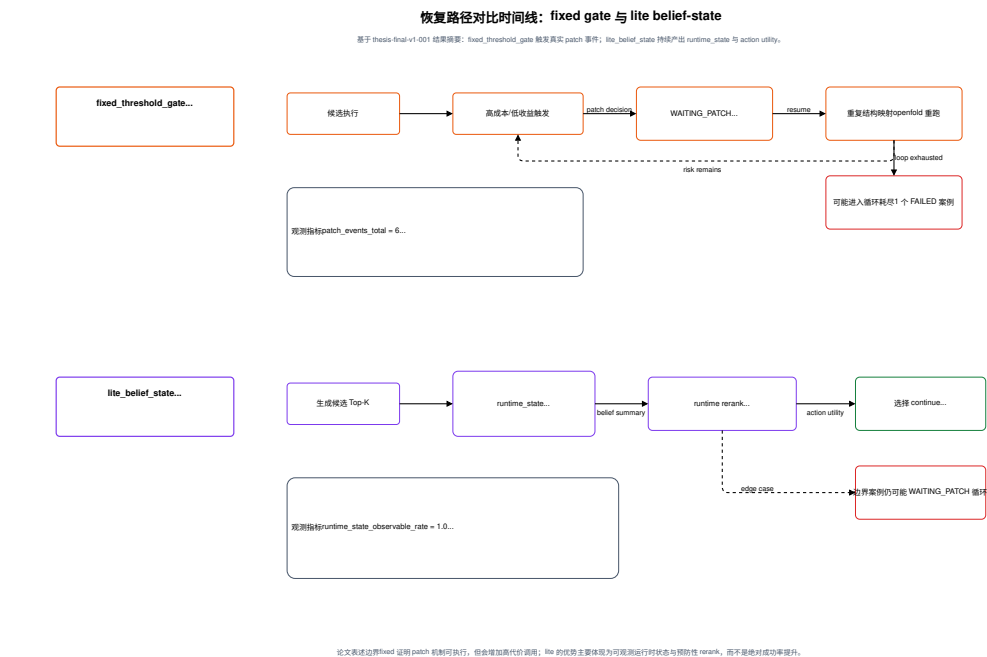


图 7-2 固定修补与 Lite belief-state 重排序的恢复路径对比

表 7-7 失败案例归因表

| 策略组                  | 任务                       | repeat | 错误类型                         | 根因摘要                                        | 证据意义                             |
|----------------------|--------------------------|--------|------------------------------|---------------------------------------------|----------------------------------|
| fixed_threshold_gate | antiquitin_sequence_eval | 02     | auto decision loop exhausted | 固定阈值门控触发 WAIT-ING_PATCH 循环, 局部修补后仍触发相同门控条件。 | 说明固定阈值门控可以触发恢复, 也可能引入额外循环和高代价调用。 |



表 7-7 (续表)

| 策略组                     | 任务                                     | repeat | 错误类型                           | 根因摘要                                                                         | 证据意义                                      |
|-------------------------|----------------------------------------|--------|--------------------------------|------------------------------------------------------------------------------|-------------------------------------------|
| lite_belief_state       | t2_ubiquitin_sequence_eval             | auto   | decision                       | 多次 WAIT-ING_PATCH 循环中 bud-get_pressure 升至 1.5, runtime adjustment 已产生但未打破循环。 | 说明 belief-state 链路可观测, 且该机制仍受候选质量和循环控制限制。 |
| dynamic_no_belief_state | t3_belief_state_stability_optimization | 0      | CANDIDATE_IO_candidates_BROKEN | 候选输出字段无法被下游引用, 候选验证在执行前失败。                                                   | 说明候选可执行性校验能够阻断无效 plan。                    |

三个失败案例均有 event log 和 snapshot,可作为可追溯的边界样本.fixed\_threshold\_gate 的 t2 案例解释了表 7-5 中高代价调用增加的来源; lite\_belief\_state 的 t2 案例展示了 RuntimeState 的动态演化; dynamic\_no\_belief\_state 的 t3 案例对应表 7-2 中 schema\_valid\_rate=0.9524。

7.8 证据产物与本章结论

表 7-8 汇总本章使用的实验产物。所有核心结论均可回溯到矩阵报告、CSV 聚合、run 级结果、事件日志或快照。

表 7-8 实验结果证据产物索引

| 证据产物    | 路径/来源                                                                                                   | 支撑内容                                          | 使用位置    |
|---------|---------------------------------------------------------------------------------------------------------|-----------------------------------------------|---------|
| 实验结果报告  | ../thesis-project.84-w/mocs/experiment/thesis-final-v1-results.md                                       | 8.4 章节概况、主结果、分层结果、失败案例                        | 全章      |
| 实验设计文档  | ../thesis-project.dev/06-experiment/final-7.2-thesis-experiment-des                                     | 研究问题、任务类、指标体系                                 | 7.1、7.2 |
| 分组映射文档  | ../thesis-project.dev/policy/experiment/algorithm-group-paper-mappi                                     | 代码 policy 映射与论文组名对应关系                         | 7.1     |
| 主指标汇总   | matrix_metrics_summary.csv                                                                              | 四组主指标、成功率、恢复、耗时和高代价调用                         | 7.3、7.6 |
| 机制增量表   | mechanism_incremental_delta.csv                                                                         | 相邻策略组 paired delta                            | 7.4     |
| 高代价调用表  | high_cost_breakdown.csv                                                                                 | 高代价调用总数和规则命中                                  | 7.5     |
| 恢复事件表   | patch_replan_breakdown/action_distribution.csv                                                          | patch/replan/action 分布                        | 7.5     |
| run 级结果 | run_level_results.json                                                                                  | 失败案例、RuntimeState、tool usage、artifact linkage | 7.6、7.7 |
| 事件日志与快照 | data/logs/thesis-final-06-experiment-001.jsonl<br>data/snapshots/thesis-final-06-experiment-001_*.jsonl | 决策链-恢复链、运行时状态和失败追踪                            | 7.7     |

本章实验得到四点结论。在当前实验矩阵中,CEBRA-WP 的机制链路可执行、可追踪, lite\_belief\_state 组 21/21 runs 产生 RuntimeState, runtime\_state\_observable\_rate 为 1.0000.fixed\_threshold\_gate 触发 6 次真实局部修补,高代价调用总数达到 28,说明固定阈值门控能够暴露恢复需求,同时也会带来额外执行成本.dynamic\_no\_belief\_state 与 lite\_belief\_state 的 high\_cost\_call\_mean 均为 0.9524, 低于 fixed\_threshold\_gate 的 1.3333; lite 的主要增量体现在 Lite belief-state / 轻量信念状态、预算压力和动作效用的可观测性。3 个 FAILED run 均有可追溯事件链, 失败集中在 medium/standard 层, 主要反映候选验证、局部修补循环和运行时控制的边界条件。

在当前 thesis-final-v1-001 设置下, 实验支持“CEBRA-WP 机制已实现且可观测”“固定阈值门控恢复存在额外成本”“Lite belief-state / 轻量信念状态提供运行时决策解释信息”等结论。成功率方面, static\_top1 为 1.0000, 其余三组为 0.9524, 最终成功率提升并非本章的主要实验结论。

第八章将结合上述实验结论与边界, 总结本文贡献并讨论后续改进方向。

## 第 8 章 总结与展望

本文围绕“基于大模型驱动的 agent 协作新一代蛋白质设计系统开发”这一题目, 面向蛋白质设计 workflows 中的工具异构、高代价调用、运行时失败和人工审查需求, 设计并实现了一个以多 Agent 协作、工具知识约束和恢复自适应规划为核心的科研工作流系统。系统将已有蛋白质设计工具和远程模型服务组织为可执行、可恢复、可审计的工作流, 并提出约束与证据感知、信念引导、恢复自适应 workflow 规划 (Constraint- and Evidence-aware Belief-guided Recovery-adaptive Workflow Planning, CEBRA-WP) 算法, 在约束、证据、运行时状态和恢复动作之间建立统一的决策机制。

全文先说明课题背景、蛋白质设计技术发展、MDP/POMDP 与 Agent 协作理论基础, 再依次展开需求分析、系统设计、系统实现、系统验证和策略实验。需求分析明确了任务录入、候选计划生成、工具执行、HITL、运行时恢复和结果审计等核心需求; 系统设计给出了分层架构、ProteinToolKG、FSM、HITL、核心数据契约和 CEBRA-WP 算法定义; 系统实现完成了后端 API、前端工作台、workflow 运行时、ToolAdapter、RuntimeEvaluator、快照和事件日志等模块; 系统测试通过 13 个测试用例覆盖功能正确性和工程可用性; 实验分析基于 thesis-final-v1-001 的 84-run 四组消融矩阵, 考察了静态规划、固定阈值门控、动态观测和 Lite belief-state / 轻量信念状态四类策略的行为差异。

### 8.1 论文工作总结

在系统设计方面, 本文将蛋白质设计任务抽象为受约束的工作流规划问题。任务目标、用户约束、工具能力、输入输出契约、预算信息和运行时观测共同决定候选 workflow 的生成、筛选和重排序。系统采用 FSM 作为任务生命周期的控制核心, 将 CREATED、PLANNING、RUNNING、WAITING\_\*, SUMMARIZING、DONE、FAILED 等状态组织为受控迁移路径; 采用 PendingAction/Decision 表示 HITL 决策; 采用 EventLog 和 TaskSnapshot 保存状态迁移、恢复动作、运行时状态和审计信息。

在算法设计方面, 本文提出 CEBRA-WP, 将约束感知、证据感知、Lite belief-state / 轻量信念状态和恢复自适应动作统一到 workflow 层。算法输入包括任务目标  $g$ 、约束集合  $C$ 、ProteinToolKG  $K$ 、历史状态  $h_t$ 、观测  $o_t$  和 Lite belief-state / 轻量

信念状态  $x_t$ ；算法过程包括候选生成、硬可行性筛选、静态评分、后验目标适配、RuntimeState 更新、候选重排序和恢复动作选择。Lite belief-state / 轻量信念状态包含  $p_{succ}$ 、 $p_{sf}$ 、 $r_{rec}$ 、 $c_{rem}$  和  $e_{suf}$  等状态量，用于刻画任务在运行时的成功概率、结构性失败风险、恢复余量、剩余成本和证据充分性。

在系统实现方面，本文基于 Python、FastAPI、Pydantic、React 和 TypeScript 完成原型系统。后端提供任务录入、任务生命周期、事件查询、报告查询和人工决策接口；前端工作台包含 Dashboard、Task Builder、Task Detail 和 Event Timeline 等页面；工作流运行时通过 WorkflowContext、PlanRunner、StepRunner、RuntimeEvaluator、PendingAction 和 Snapshot 等模块实现任务推进、工具调用、重试、恢复、等待态和审计记录。工具侧通过 ToolAdapter 抽象封装结构预测、序列生成、质量控制等外部能力，使异构蛋白质设计工具能够在统一契约下接入系统。

在系统验证方面，本文构建了覆盖 API、Web、CLI、FSM、HITL、快照、安全边界、失败恢复和端到端执行的验证体系。13 个测试用例中 12 个通过，1 个 CLI 相关用例部分通过。验证结果表明，在本文测试范围内，系统可以处理任务创建、候选生成、人工确认、工具执行、事件追踪和报告输出；等待态下执行停止、终态不可变、快照恢复后不自动推进等关键不变性均有测试或日志证据支撑。

在实验分析方面，本文使用 12 个 task\_keys、4 组策略、84 次运行构成消融矩阵。实验结果显示，84 runs 中 81 个进入 DONE，3 个进入 FAILED；lite\_belief\_state 组 21/21 runs 产生有效 RuntimeState，runtime\_state\_observable\_rate 为 1.0000；fixed\_threshold\_gate 组触发 6 次真实局部修补，高代价调用总数为 28；dynamic\_no\_belief\_state 与 lite\_belief\_state 的高代价调用总数均为 20，低于 fixed\_threshold\_gate 组。上述结果支持三项判断：CEBRA-WP 机制已实现且可观测，固定阈值门控恢复存在额外成本，Lite belief-state / 轻量信念状态能够提供运行时决策解释信息。

## 8.2 主要贡献

本文的第一项贡献是构建了一个面向蛋白质设计工作流的可恢复、可审计原型系统。系统围绕任务接入、工具知识约束、候选计划、执行状态、人机决策、恢复路径和审计记录建立了原型层面的工程闭环。第六章的系统验证和第七章的 84-run 实验表明，该系统能够在本文实验设置下支撑多任务、多策略的批量运行和结果追踪。

第二项贡献是提出并实现了 CEBRA-WP workflow 规划算法。该算法定位于 workflow 层的候选筛选、运行时重排序和恢复动作选择。CEBRA-WP 将 ProteinToolKG 中的工具能力、schema、输入输出、成本、风险和证据状态纳入候选评估，并通过 Lite belief-state / 轻量信念状态将运行时观测转化为可解释的决策依据。

第三项贡献是建立了从系统验证到策略消融的证据链。系统验证部分使用 TC-S01 至 TC-S13 覆盖 API、状态机、HITL、快照、恢复和安全边界；实验分析部分

使用表 7-1 至表 7-8 固定实验配置、主结果、分层结果、机制增量、成本分析、机制可观测性、失败归因和证据产物。论文中的主要结论均能回溯到测试日志、API 响应、EventLog、Snapshot、run-level result 或聚合 CSV。

第四项贡献是对 CEBRA-WP 的机制价值给出了有边界的实验分析。实验结果展示了 Lite belief-state / 轻量信念状态的 RuntimeState、budget pressure 和 action utility 的持续可观测性；同时，fixed\_threshold\_gate 的局部修补循环和额外高代价调用说明固定阈值门控恢复会带来可测成本。基于这些结果，本文将算法贡献限定在高代价 workflows 中的恢复控制、成本意识和审计解释，而不是单纯以成功率作为评价依据。

## 8.3 局限性

本文仍存在若干局限。实验规模方面，thesis-final-v1-001 包含 84 runs，每组 21 runs，可以支撑机制分析和方向性比较，但统计效力仍有限。尤其是 dynamic\_no\_belief\_state 与 lite\_belief\_state 在 success\_rate 和 high\_cost\_call\_mean 上完全相同，说明当前任务集对二者性能差异的放大能力有限。

恢复机制在矩阵实验中的覆盖也不完整。84-run 矩阵中真实局部修补仅出现在 fixed\_threshold\_gate 组，且四组均未触发真实重规划或后缀重规划。第六章 focused tests 已验证 suffix\_replan 和 terminal\_stop 路径可达，但第七章批量实验主要提供局部修补和高代价调用方面的证据。后续仍需要通过更强的失败诱导任务补足重规划、stop 和 escalation 的矩阵级证据。

第三，系统仍处于原型阶段。当前任务记录以运行时任务表和日志/快照文件为主，数据库持久化需要进一步补充；ProteinToolKG 以静态配置为主，动态工具注册和在线能力更新能力有限；前端结构区域主要提供产物入口和报告展示；远程服务的自动健康管理和故障切换仍可进一步增强。这些限制主要影响系统在长期科研环境中的部署能力。

第四，本文实验以内部消融为主，外部基线对比仍有扩展空间。当前四组策略能够分离静态选择、固定阈值门控、动态观测和 Lite belief-state / 轻量信念状态的机制差异；后续可与 ReAct-style、Tree-of-Thought-style 或 Reflexion-style 等通用 Agent 方法在同一任务集上进行系统对照。外部对照可以进一步说明 CEBRA-WP 在结构化约束、HITL、恢复审计和高代价控制方面的相对价值。

## 8.4 未来工作展望

后续工作应进一步扩大实验规模并增强压力任务设计。未来实验可增加任务种类、repeat 数和失败诱导条件，例如构造更强的工具不可用、预算冲突、schema 错误、I/O 闭包错误和安全约束冲突场景，使 patch\_local、suffix\_replan、terminal\_stop

和 safety block 在批量矩阵中均能被触发。这样可以将当前以 focused tests 为主的恢复路径验证扩展为统计层面的恢复能力分析。

外部 Agent 基线和蛋白质设计前沿方法对比也值得补充。ReAct、Tree of Thoughts 和 Reflexion 等通用 Agent 方法适合构成 workflow 决策层基线；ProteinGuide、ProteinZero 等近期预印本则提示了蛋白质生成中属性引导、在线反馈和自改进方向<sup>[28,29]</sup>。未来可以将 CEBRA-WP 与这些方向结合，比较结构化 workflow 控制、属性引导生成和在线反馈优化之间的关系。

恢复策略还需要更明确的 escalation 机制。当前局部修补循环耗尽后进入 FAILED，说明系统能够识别恢复失败，但仍缺少从局部修补到后续重规划再到终止止损的自动升级策略。未来可在 RuntimeEvaluator 中引入恢复次数、重复失败类型、预算压力和候选多样性等因素，当同类局部修补多次失败时自动提升到 suffix\_replan 或 stop 候选，从而降低循环恢复带来的时间和高代价调用开销。

原型系统的工程化演进同样重要。任务状态可以迁移到关系数据库或文档数据库；ProteinToolKG 可以迁移到图数据库或可热更新的服务；事件日志、快照和实验产物可以统一纳入实验追踪平台；前端可以补充更完整的结构可视化和候选对比视图；远程模型服务可以加入自动探活、降级切换和调用配额管理。这些改进有助于提升系统在真实科研工作流中的可维护性和长期运行能力。

从更长期的方向看，CEBRA-WP 的工作流层思想可扩展到蛋白质设计之外的科学计算场景。结构化候选生成、硬约束筛选、运行时状态估计、恢复动作选择和审计追踪，同样适用于分子动力学模拟、材料筛选、基因组分析和其他高代价科学 workflow。面向更广泛任务建立标准化 benchmark 和可复现实验套件，例如参考结构设计基准中对任务、指标和证据的组织方式<sup>[30]</sup>，是进一步验证该类恢复自适应 workflow 规划方法的重要方向。

## 参考文献

- [1] LEAVER-FAY A, TYKA M, LEWIS S M, et al. ROSETTA3: an object-oriented software suite for the simulation and design of macromolecules[M/OL].//Methods in Enzymology: Vol. 487. Academic Press, 2011: 545-574. <https://doi.org/10.1016/B978-0-12-381270-4.00019-6>.
- [2] JUMPER J, EVANS R, PRITZEL A, et al. Highly accurate protein structure prediction with AlphaFold[J/OL]. Nature, 2021, 596: 583-589. <https://doi.org/10.1038/s41586-021-03819-2>.
- [3] ABRAMSON J, ADLER J, DUNGER J, et al. Accurate structure prediction of

- biomolecular interactions with AlphaFold 3[J/OL]. *Nature*, 2024, 630: 493-500. <https://doi.org/10.1038/s41586-024-07487-w>.
- [4] DAUPARAS J, ANISHCHENKO I, BENNETT N, et al. Robust deep learning-based protein sequence design using ProteinMPNN[J/OL]. *Science*, 2022, 378: 49-56. <https://doi.org/10.1126/science.add2187>.
- [5] FERRUZ N, SCHMIDT S, HOCKER B. ProtGPT2 is a deep unsupervised language model for protein design[J/OL]. *Nature Communications*, 2022, 13: 4348. <https://doi.org/10.1038/s41467-022-32007-7>.
- [6] LIN Z, AKIN H, RAO R, et al. Evolutionary-scale prediction of atomic-level protein structure with a language model[J/OL]. *Science*, 2023, 379: 1123-1130. <https://doi.org/10.1126/science.ade2574>.
- [7] WATSON J L, JUERGENS D, BENNETT N R, et al. De novo design of protein structure and function with RFdiffusion[J/OL]. *Nature*, 2023, 620: 1089-1100. <https://doi.org/10.1038/s41586-023-06415-8>.
- [8] YAO S, ZHAO J, YU D, et al. ReAct: synergizing reasoning and acting in language models[A/OL]. *arXiv* (2022). <https://arxiv.org/abs/2210.03629>. DOI: 10.48550/arXiv.2210.03629.
- [9] SCHICK T, DWIVEDI-YU J, DESSI R, et al. Toolformer: language models can teach themselves to use tools[A/OL]. *arXiv* (2023). <https://arxiv.org/abs/2302.04761>. DOI: 10.48550/arXiv.2302.04761.
- [10] AHDRITZ G, BOUATTA N, FLORISTEAN C, et al. OpenFold: retraining AlphaFold2 yields new insights into its learning mechanisms and capacity for generalization[J/OL]. *Nature Methods*, 2024, 21: 1514-1524. <https://doi.org/10.1038/s41592-024-02272-z>.
- [11] DEELMAN E, SINGH G, SU M H, et al. Pegasus: a framework for mapping complex scientific workflows onto distributed systems[J/OL]. *Scientific Programming*, 2005, 13(3): 219-237. <https://doi.org/10.1155/2005/128026>.
- [12] DI TOMMASO P, CHATZOU M, FLODEN E W, et al. Nextflow enables reproducible computational workflows[J/OL]. *Nature Biotechnology*, 2017, 35: 316-319. <https://doi.org/10.1038/nbt.3820>.

- [13] BELLMAN R. A markovian decision process[J/OL]. Indiana University Mathematics Journal, 1957, 6(4): 679-684. <https://doi.org/10.1512/iumj.1957.6.56038>.
- [14] PUTERMAN M L. Markov decision processes: discrete stochastic dynamic programming[M/OL]. Wiley, 1994. <https://doi.org/10.1002/9780470316887>.
- [15] SONDIK E J. The optimal control of partially observable markov processes over the infinite horizon: discounted costs[J/OL]. Operations Research, 1978, 26(2): 282-304. <https://doi.org/10.1287/opre.26.2.282>.
- [16] MONAHAN G E. State of the art—A survey of partially observable markov decision processes: theory, models, and algorithms[J/OL]. Management Science, 1982, 28(1): 1-16. <https://doi.org/10.1287/mnsc.28.1.1>.
- [17] KAEHLBLING L P, LITTMAN M L, CASSANDRA A R. Planning and acting in partially observable stochastic domains[J/OL]. Artificial Intelligence, 1998, 101(1–2): 99-134. [https://doi.org/10.1016/S0004-3702\(98\)00023-X](https://doi.org/10.1016/S0004-3702(98)00023-X).
- [18] COCK P J A, ANTAO T, CHANG J T, et al. Biopython: freely available python tools for computational molecular biology and bioinformatics[J/OL]. Bioinformatics, 2009, 25(11): 1422-1423. <https://doi.org/10.1093/bioinformatics/btp163>.
- [19] YAO S, YU D, ZHAO J, et al. Tree of thoughts: deliberate problem solving with large language models[A/OL]. arXiv (2023). <https://arxiv.org/abs/2305.10601>. DOI: 10.48550/arXiv.2305.10601.
- [20] SHINN N, CASSANO F, BERMAN E, et al. Reflexion: language agents with verbal reinforcement learning[A/OL]. arXiv (2023). <https://arxiv.org/abs/2303.11366>. DOI: 10.48550/arXiv.2303.11366.
- [21] XIE T, ZHANG D, CHEN J, et al. OSWorld: benchmarking multimodal agents for open-ended tasks in real computer environments[A/OL]. arXiv (2024). <https://arxiv.org/abs/2404.07972>. DOI: 10.48550/arXiv.2404.07972.
- [22] SHANI G. Heuristics for partially observable stochastic contingent planning[A/OL]. arXiv (2024). <https://arxiv.org/abs/2410.05870>. DOI: 10.48550/arXiv.2410.05870.
- [23] CARRARA N, LEURENT E, LAROCHE R, et al. Budgeted reinforcement learning in continuous state space[C/OL]//Advances in Neural Information Processing Systems. 2019. <https://arxiv.org/abs/1903.01004>. DOI: 10.48550/arXiv.1903.01004.



- [24] KSHIRSAGAR M, NIE A, CHENG C A, et al. RosettaSearch: multi-objective inference-time search for protein sequence design[A/OL]. arXiv (2026). <https://arxiv.org/abs/2604.17175>. DOI: 10.48550/arXiv.2604.17175.
- [25] GE F, ZHU J, ZHANG L, et al. AutoBinder agent: an MCP-based agent for end-to-end protein binder design[A/OL]. arXiv (2026). <https://arxiv.org/abs/2602.00019>. DOI: 10.48550/arXiv.2602.00019.
- [26] HOU X, LIU J, SHI C, et al. Property-driven protein inverse folding with multi-objective preference alignment[A/OL]. arXiv (2026). <https://arxiv.org/abs/2603.06748>. DOI: 10.48550/arXiv.2603.06748.
- [27] SIMMHAN Y, BARGA R, VAN INGEN C, et al. Reliable management of community data pipelines using scientific workflows: MSR-TR-2009-125[R/OL]. Microsoft Research, 2009. <https://www.microsoft.com/en-us/research/publication/reliable-management-of-community-data-pipelines-using-scientific-workflows/>.
- [28] XIONG J, GAUR I, LUKARSKA M, et al. ProteinGuide: on-the-fly property guidance for protein sequence generative models[A/OL]. arXiv (2025). <https://arxiv.org/abs/2505.04823>. DOI: 10.48550/arXiv.2505.04823.
- [29] WANG Z, FAN J, GUO R, et al. ProteinZero: self-improving protein generation via online reinforcement learning[A/OL]. arXiv (2025). <https://arxiv.org/abs/2506.07459>. DOI: 10.48550/arXiv.2506.07459.
- [30] WANG C, ZHONG B, ZHANG Z, et al. PDB-struct: a comprehensive benchmark for structure-based protein design[A/OL]. arXiv (2023). <https://arxiv.org/abs/2312.00080>. DOI: 10.48550/arXiv.2312.00080.